Department of Computer Science and Engineering

B.Tech III Semester

Academic Year 2026-2027

DATA ANALYSIS ESSENTIALS

SALES DATA ANALYSIS SYSTEM



ADITYA UNIVERSITY

Surampalem, Kakinada District, Andhra Pradesh-533437

Submitted by

M. JAYA KEERTHI	(25B11CS351)
N. RUKMINI	(25B11CS637)
N. SUNIL	(25B11CS642)
K. AKSHAY	(25B11CS446)

ADITYA UNIVERSITY

School of Computing

Department of Computer Science and Engineering

DATA ANALYSIS ESSENTIALS

WEB SCRAPING AND PRODUCT PRICE ANALYSIS SYSTEM

Submitted by

M. JAYA KEERTHI	(25B11CS351)
N. RUKMINI	(25B11CS637)
N. SUNIL	(25B11CS642)
K. AKSHAY	(25B11CS446)

Under the guidance of

Dr. T. Sudha Rani

Assistant Professor, Department of CSE

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering

ADITYA UNIVERSITY

Surampalem

Academic Year 2026–2027

ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Data Analysis and Essentials (DAE) project work entitled **"Web Scraping and Product Price Analysis System"** is submitted by

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2026-2027.

M. JAYA KEERTHI	(25B11CS351)
N. RUKMINI	(25B11CS637)
N. SUNIL	(25B11CS642)
K. AKSHAY	(25B11CS446)

Project Guide
Dr. T. Sudha Rani,
Assistant Professor,
Department of Computer Science and Engineering

Head of the Department
Dr. A. Phani Sridhar, Ph.D,
Associate Professor,
Department of Computer Science and Engineering

DECLARATION

We hereby declare that the Data Analytics and Exploration project entitled **"Web Scraping and Product Price Analysis System"** is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem

Date:

By

M. JAYA KEERTHI	(25B11CS351)
N. RUKMINI	(25B11CS637)
N. SUNIL	(25B11CS642)
K. AKSHAY	(25B11CS446)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Data Analytics and Exploration Project Guide, **Dr. T. Sudha Rani, Assistant Professor**, who has guided us throughout the project. Valuable support, encouragement and guidance were extended to us for the successful completion of this project.

We would like to thank our Deputy Heads of the Department**, Dr. K. Chandra Sekhar, Professor, Dr. Hari Jyothula, Professor and Dr. P. Siva Satya Prasad, Professor**, Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. A. Phani Sridhar, Associate Professor**, for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean**, School of Engineering, for his encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, and Dr. M. Sreenivasa Reddy, Deputy Chancellor and Management**, Aditya University, for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the faculty members, lab technicians, non-teaching staff, and our friends who have directly or indirectly supported us in completing this project successfully.

ABSTRACT

The Web Scraping and Product Price Analysis System is a Data Analytics and Exploration (DAE) project that demonstrates a complete, end-to-end workflow for collecting, cleaning, analysing and visualising product data using Python. Using Playwright and a headless Chromium browser in Google Colab, the system scrapes 500 product records, each with ten attributes, from all 20 pages of the Data Table page of Scraping Sandbox (https://scrapingsandbox.com/data-table), a practice website designed for learning web scraping; no commercial e-commerce website is scraped. The records are stored in a Pandas DataFrame, validated for size, missing values and duplicate rows, and cleaned by converting the Price, Rating and Variants columns to numeric form. The cleaned dataset is saved as CSV and Excel files.

The analysis stage calculates the average, minimum, maximum and median price, identifies the cheapest and the most expensive products, and compares prices across categories and vendors. Products are classified into Low, Medium and High price groups, and the correlation between price and rating is measured. Five Matplotlib charts present the results visually, and the final summary and analysis tables are exported as CSV files. The project uses no machine-learning model and does not predict future prices; its purpose is to give a clear, reproducible demonstration of web scraping and exploratory price analysis.

TABLE OF CONTENTS

S.No.	Contents	Pages
1	Title Page	ii
2	Certificate	iii
3	Declaration	iv
4	Acknowledgement	v
5	Abstract	vi
6	Table of contents	vii
7	List of Figures	xi
8	List of Tables	xii
9	Acronyms	xiii
Chapter 1	**Introduction**	**1**
1.1	Introduction to Data Analytics	1
1.2	Introduction to Web Scraping	1
1.3	Introduction to Product Price Analysis	1
1.4	Project Overview	2
1.5	Problem Statement	2
1.6	Objectives of the Project	2
1.7	Scope of the Project	3
1.8	Need for the Project	3
1.9	Organisation of the Report	4
Chapter 2	**Dataset Description and Understanding**	**5**
2.1	Dataset Source	5
2.2	Description of Scraping Sandbox	5
2.3	Method of Data Collection	5
2.4	Dataset Structure	5
2.5	Dataset Attributes	5
2.6	Dataset Dimensions	6
2.7	Types of Data	6
2.8	Initial Dataset Exploration	6
Chapter 3	**Data Collection and Preprocessing**	**8**
3.1	Overview of Data Preprocessing	8
3.2	Installing Required Libraries	8
3.3	Installing Chromium	8
3.4	Testing the Browser	9
3.5	Opening the Target Website	9
3.6	Pagination and Product Collection	10
3.7	Extracting Table Rows	10

S.No.	Contents	Pages
3.8	Extracting Table Cells	11
3.9	Creating the Pandas DataFrame	12
3.10	Dataset Validation	12
3.11	Handling Missing Values	13
3.12	Checking Duplicate Records	13
3.13	Cleaning the Price Column	13
3.14	Converting Rating and Variants	14
3.15	Verifying Data Types	14
3.16	Saving the Cleaned Dataset	14
Chapter 4	**System Design and Methodology**	**16**
4.1	Proposed Methodology	16
4.2	System Architecture	16
4.3	Project Workflow	17
4.4	Flowchart	18
4.5	Tools and Technologies	20
4.6	Algorithms and Functions Used	20
4.7	Functional Modules	21
Chapter 5	**Implementation**	**23**
5.1	Development Environment	23
5.2	Libraries Used	23
5.3	Web Scraping Implementation	23
5.4	DataFrame Creation	23
5.5	Data Validation Implementation	23
5.6	Data Cleaning Implementation	24
5.7	Dataset Saving and Download	24
5.8	Loading the Dataset for Analysis	24
5.9	Price Column Preparation	25
5.10	Basic Price Analysis	25
5.11	Complete Price Statistics	26
5.12	Cheapest Product Identification	26
5.13	Most Expensive Product Identification	27
5.14	Category-wise Price Analysis	27
5.15	Top 10 Expensive Products	28
5.16	Top 10 Cheapest Products	28
5.17	Price Range Classification	28
5.18	Price-Rating Relationship	29
5.19	Price-Rating Correlation	30
5.20	Vendor-wise Average Price	30

S.No.	Contents	Pages
5.21	Final Summary Table	31
5.22	Saving Analysis Results	31
Chapter 6	**Data Visualization and Results**	**33**
6.1	Introduction to Data Visualisation	33
6.2	Product Price Distribution Histogram	33
6.3	Average Price by Category Bar Chart	34
6.4	Product Price Range Chart	35
6.5	Product Price vs Rating Scatter Plot	36
6.6	Vendor-wise Average Price Bar Chart	36
6.7	Interpretation of Visualisations	37
Chapter 7	**Analysis of Results and Discussion**	**38**
7.1	Overview of Analysis	38
7.2	Overall Price Statistics	38
7.3	Cheapest and Most Expensive Products	38
7.4	Category-wise Price Comparison	39
7.5	Price Range Distribution	40
7.6	Price-Rating Correlation	40
7.7	Vendor-wise Price Comparison	41
7.8	Discussion of Findings	41
7.9	Project Outcomes	42
Chapter 8	**Output Screenshots**	**43**
8.1	Scraping Sandbox Website Screenshot	43
8.2	Browser Test Output	43
8.3	Total Products Scraped Output	44
8.4	DataFrame Preview	44
8.5	Dataset Validation Output	45
8.6	Data Cleaning Output	45
8.7	Data Types Output	45
8.8	Dataset Saving Output	46
8.9	Price Statistics Output	46
8.10	Cheapest Product Output	47
8.11	Most Expensive Product Output	47
8.12	Category Price Analysis Output	47
8.13	Top 10 Expensive Products Output	48
8.14	Top 10 Cheapest Products Output	48
8.15	Price Range Analysis Output	49
8.16	Correlation Output	49
8.17	Vendor Price Analysis Output	49

S.No.	Contents	Pages
8.18	Final Summary Table	49
8.19	Visualisation Screenshots	50
Chapter 9	**Conclusion, Limitations and Future Enhancements**	**51**
9.1	Conclusion	51
9.2	Achievements	51
9.3	Limitations	51
9.4	Future Enhancements	52
Chapter 10	**References**	**53**
10.1	References	53
Chapter 11	**Appendix — Source Code**	**54**
11.1	Part A: Web Scraping and Dataset Creation	54
11.2	Part B: Product Price Analysis	57

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	System architecture of the Web Scraping and Product Price Analysis System	17
2	Project workflow from the Scraping Sandbox website to the exported results	18
3	Flowchart of the scraping, preprocessing and analysis process	19
4	Output of the dataset-loading step: number of products, number of columns and df.head()	25
5	Product Price Distribution histogram	33
6	Average Product Price by Category	34
7	Products by Price Range	35
8	Product Price vs Rating	36
9	Average Product Price by Vendor	37
10	The Data Table page of Scraping Sandbox (scrapingsandbox.com/data-table)	43
11	Cell 3: browser test code and output "Browser started successfully!"	44
12	Cell 4 output: 25 rows found on each page and TOTAL PRODUCTS SCRAPED: 500	44
13	Cell 5 output: the first five rows of the DataFrame	45
14	Cell 6 output: rows, columns, column names, missing values and duplicate rows	45
15	Cell 7 output: the DataFrame after cleaning Price, Rating and Variants (screenshot placeholder)	45
16	Cell 8 output: data types of all columns after cleaning (screenshot placeholder)	46
17	Cell 9 output: dataset saved to CSV and Excel files	46
18	Part B, step 3: average, minimum, maximum and median price	46
19	Part B, step 4: output of df["Price"].describe()	46
20	Part B, step 5: cheapest product	47
21	Part B, step 6: most expensive product	47
22	Part B, step 8: average price by category	47
23	Part B, step 10: minimum, maximum and mean price by category	48
24	Part B, step 11: top 10 most expensive products	48
25	Part B, step 12: top 10 cheapest products	48
26	Part B, step 13 output: number of products in each price range (screenshot placeholder)	49
27	Part B, step 16: correlation between price and rating	49
28	Part B, step 17: average price by vendor	49
29	Part B, step 19: final price analysis summary table	50
30	Visualisation gallery of the five charts generated by the analysis code	50

LIST OF TABLES

Table No.	Table Name	Page No.
1	Organisation of the report	4
2	Attributes of the dataset	6
3	First five records of the scraped DataFrame (as displayed by df.head())	7
4	Tools and technologies used in the project	20
5	Approaches to collecting data from web pages	20
6	Main functions used in the implementation	21
7	Libraries imported by the code	23
8	Overall price statistics (output of describe() and the summary table)	38
9	Cheapest and most expensive product	38
10	Top 10 most expensive products (the first column is the DataFrame row index)	39
11	Top 10 cheapest products (the first column is the DataFrame row index)	39
12	Category-wise price statistics, sorted by mean price (from the average-price and min/max/mean outputs)	40
13	Approximate limits of the three price groups (derived from the minimum and maximum price)	40
14	Vendor-wise average price (from the average-price-by-vendor output)	41

ACRONYMS

Acronym	Expansion
DAE	Data Analytics and Exploration
EDA	Exploratory Data Analysis
API	Application Programming Interface
CSV	Comma-Separated Values
XLSX	Microsoft Excel Open XML Spreadsheet
HTML	HyperText Markup Language
DOM	Document Object Model
URL	Uniform Resource Locator
DF	DataFrame (Pandas)
SKU	Stock Keeping Unit
NaN	Not a Number (missing value)
IDE	Integrated Development Environment
CPU	Central Processing Unit

CHAPTER 1
INTRODUCTION

1.1 Introduction to Data Analytics

Data analytics is the process of examining raw data in order to discover useful patterns, summarise information and support decisions. Modern organisations, websites and applications generate data continuously, but raw data is rarely ready for use. It may be stored in an unsuitable format, it may contain missing or incorrectly typed values, and it is usually too large to be understood by simply reading it. Data analytics therefore follows a sequence of steps: collecting the data, checking and cleaning it, exploring it with statistics, and presenting the findings with tables and charts.

Exploratory Data Analysis (EDA) is the part of this sequence in which the analyst studies a dataset to understand its structure and behaviour before drawing conclusions. Typical EDA activities include calculating the mean, median, minimum and maximum of a numeric column, grouping records by a category and comparing the groups, counting how many records fall into each class, and drawing histograms or scatter plots. This project applies exactly these EDA techniques to a collection of product records using the Python programming language and its data-analysis libraries.

1.2 Introduction to Web Scraping

Web scraping is the automated extraction of information that is displayed on web pages. A person who wants to collect product details from a website can copy them manually, but this is slow, error-prone and impractical when hundreds of records are spread over many pages. A scraping program does the same work automatically: it opens the page, finds the elements that contain the required text, reads that text and stores it in a structured form such as a list or a table.

Many modern web pages do not place all of their data in a single static page. A table may be divided into pages that are changed by clicking numbered buttons, and the content is updated without loading a new address. For such pages a browser-automation tool is useful because it behaves like a real browser: it can wait for a table to appear, click a button and read the updated content. This project uses **Playwright**, which controls a **Chromium** browser from Python, for this purpose. The browser runs in headless mode, which means that it works in the background without a visible window.

1.3 Introduction to Product Price Analysis

Price is one of the most important attributes of a product. Product price analysis studies how prices are distributed across a collection of products: what the average and median prices are, how widely prices vary, which products are the cheapest and the most expensive, how prices differ between categories and vendors, and whether the price of a product appears to be related

to its rating. Such an analysis helps a reader to understand a product collection quickly, without reading every record.

In this project the analysis is performed on the data that was actually scraped from the practice website. It is descriptive in nature: it summarises and compares the available records. It does not forecast future prices and it does not recommend products.

1.4 Project Overview

The Web Scraping and Product Price Analysis System is a Python project developed in Google Colab. It consists of two connected parts. **Part A – Web Scraping and Dataset Creation** installs Playwright and Chromium, scrapes 20 pages of the Scraping Sandbox Data Table, converts the 500 collected product records into a Pandas DataFrame, validates and cleans the data, and saves it as CSV and Excel files. **Part B – Product Price Analysis** reloads the saved CSV file and performs the statistical, category-wise, vendor-wise, price-range and correlation analyses, draws five charts, builds a final summary table and exports the analysis tables as CSV files.

Web scraping, data cleaning, exploratory analysis, statistical summaries and visualisation work together as a pipeline. Scraping produces the raw records; cleaning turns text such as "$155.62" into numbers that can be calculated; the statistical summaries describe the numbers; the groupings compare categories and vendors; and the charts make these results easy to see. Each stage depends on the output of the previous one, which is why the project is documented in the same order.

1.5 Problem Statement

Product websites contain a large amount of information for each item, such as the product name, vendor, category, price, rating, stock status, SKU and number of variants. Collecting this information manually from many pages is time-consuming and difficult to organise. Prices differ between categories and vendors, which makes manual comparison difficult, and data taken directly from a web page is stored as text that must be cleaned and structured before it can be analysed. There is therefore a need for an automated approach that collects the product data, prepares it for analysis and summarises the price-related information clearly.

1.6 Objectives of the Project

- To automate the collection of product records from the Scraping Sandbox Data Table using Playwright and Chromium.
- To collect the records from all 20 pages of the table, with 25 rows per page, giving a target of 500 records.
- To convert the scraped records into a structured Pandas DataFrame with ten named columns.
- To validate the dataset by checking its size, column names, missing values and duplicate rows.

- To clean the Price column and convert Price, Rating and Variants into numeric formats.
- To calculate price statistics and to identify the cheapest and the most expensive product.
- To compare prices across categories and vendors and to classify products into Low, Medium and High price groups.
- To examine the relationship between price and rating using a scatter plot and a correlation coefficient.
- To present the results with Matplotlib charts and to export the cleaned data and analysis results to CSV and Excel files.

1.7 Scope of the Project

The scope of the project is limited to the data available on the Scraping Sandbox Data Table page. The implemented analysis covers the price, rating, category and vendor attributes of the 500 records. The project is implemented as a set of Google Colab notebook cells; it has no separate user interface. The following are **not** part of the implemented system: discount analysis, analysis of customer-review text, machine learning, product recommendation, price prediction, an interactive dashboard and scraping of any commercial e-commerce website.

1.8 Need for the Project

Manual collection and comparison of product data does not scale. A program can visit all pages in the same way every time, which removes copying mistakes and saves effort. Cleaning and analysing the data in Python makes the work repeatable: the same cells can be run again to reproduce the same dataset structure and the same analysis. The project also gives the team practical experience of the complete data-analytics life cycle, from raw web data to summarised, visualised results, using a website that is intended for practising scraping.

1.9 Organisation of the Report

Chapter	Title	Content
1	Introduction	Background, problem statement, objectives, scope and need.
2	Dataset Description and Understanding	Data source, collection method, attributes, dimensions and initial exploration.
3	Data Collection and Preprocessing	Scraping code, DataFrame creation, validation, cleaning and saving.
4	System Design and Methodology	Architecture, workflow, flowchart, technologies, functions and modules.
5	Implementation	Explanation of the analysis code, step by step.
6	Data Visualisation and Results	The five charts and their interpretation.
7	Analysis of Results and Discussion	Numerical findings, discussion and project outcomes.
8	Output Screenshots	Screenshots of the notebook outputs.
9	Conclusion, Limitations and Future Enhancements	Summary, achievements, limitations and future work.
10	References	Documentation and sources used.
11	Appendix – Source Code	Complete source code of Part A and Part B.

Table 1. Organisation of the report

CHAPTER 2
DATASET DESCRIPTION AND UNDERSTANDING

2.1 Dataset Source

The dataset used in this project is the product table available at **https://scrapingsandbox.com/data-table**. It was not downloaded from a file; it was collected directly from the web page by the scraping code described in Chapter 3. The dataset therefore represents exactly the records that were available on that page when the code was run.

2.2 Description of Scraping Sandbox

Scraping Sandbox is a practice website that provides pages for learning and testing web scraping. The Data Table page presents product information in a table whose rows are divided into pages. It was chosen for this project because it is meant to be scraped, it contains a reasonable number of records with several useful attributes (text, price, rating and category information), and it allows the complete scraping and analysis workflow to be demonstrated without collecting data from a commercial marketplace. It is important to note that this is a practice website. The project does not scrape Amazon, Flipkart or any other commercial e-commerce site.

2.3 Method of Data Collection

The data is collected with Playwright, which launches a headless Chromium browser from Python. The browser opens the Data Table page and waits until the table is available. The code then loops through 20 pages. For every page after the first, it clicks the corresponding page-number button and waits briefly for the table to update. On each page the code reads every row of the table body and every cell inside the row. Each page contains 25 rows, so 20 pages give the intended total of 500 product records. The scraping output shown in Chapter 8 reports "Rows found: 25" for the pages and a final total of 500 products.

2.4 Dataset Structure

Every scraped row is stored as a list of ten text values, and the list of all rows is converted into a Pandas DataFrame. The DataFrame is a two-dimensional table in which each row is one product and each column is one attribute. The columns are assigned names in the code in the order in which the values appear on the web page.

2.5 Dataset Attributes

The DataFrame contains ten columns. Table 2 describes each attribute and gives its data type after preprocessing, as supported by the cleaning code. Only Price, Rating and Variants are explicitly converted; the remaining columns keep the text values that were read from the page.

No.	Attribute	Description	Data type after preprocessing
1	ID	Serial number of the product record on the website.	Text (object) after scraping
2	Image	Image cell of the table row. It has no text, so the scraped value is empty.	Text (object), empty values
3	Product_Name	Name of the product.	Text (object)
4	Vendor	Name of the vendor that supplies the product.	Text (object)
5	Category	Category to which the product belongs.	Text (object)
6	Price	Price of the product. Converted to a number by removing "$" and ",".	float
7	Rating	Customer rating of the product, converted with pandas.to_numeric().	Numeric (float)
8	Stock	Stock status of the product, shown as Yes or No.	Text (object)
9	SKU	Stock Keeping Unit code of the product (for example SKU-HEA-0001).	Text (object)
10	Variants	Number of variants of the product, converted with pandas.to_numeric().	Numeric

Table 2. Attributes of the dataset

When the saved CSV file is reloaded in Part B, Pandas infers the types of the remaining columns again; for example, the ID column is read as an integer and the empty Image column is read as missing (NaN) values. The analysis only uses the Price, Rating, Category, Vendor and Product_Name columns.

2.6 Dataset Dimensions

The scraping code is intended to collect 500 records (20 pages × 25 rows) with 10 columns, so the DataFrame shape is expected to be (500, 10). The notebook output shown in Chapter 8 confirms this: "Dataset shape: (500, 10)", "Number of rows: 500" and "Number of columns: 10".

2.7 Types of Data

The dataset contains three kinds of data. **Numerical data** consists of Price, Rating and Variants, which can be used in calculations. **Categorical data** consists of Vendor, Category and Stock, which place each product in a group and are used for grouping and comparison. **Identifier and descriptive text** consists of ID, Product_Name, SKU and the empty Image column, which identify or describe a record but are not used in the statistical calculations. Price is a continuous numerical variable, whereas Rating and Variants are numeric values that are used as supplied.

2.8 Initial Dataset Exploration

Immediately after the DataFrame is created, the first rows are displayed with df.head(), and the dataset is validated with the checks in Cell 6 (Section 3.10). The validation output supplied in

the notebook shows 500 rows and 10 columns, the ten column names listed in Section 2.5, a missing-value count of 0 for every column, and 0 duplicate rows. The first five records displayed by df.head() are reproduced in Table 3. The Price values still contain the dollar symbol at this stage because the cleaning step has not yet been executed.

ID	Product_Name	Vendor	Category	Price	Rating	Stock	SKU	Variants
1	Lightweight Probiotics	SoundWave	Health	$155.62	4	Yes	SKU-HEA-0001	4
2	Wireless LED Desk Lamp	SoundWave	Electronics	$145.40	4.6	No	SKU-ELE-0002	2
3	Deluxe Tea Collection	StyleCraft	Food & Beverage	$185.01	3.5	Yes	SKU-FOO-0003	3
4	Essential Tire Gauge	TechVault	Automotive	$70.86	3.6	No	SKU-AUT-0004	5
5	Smart Carving Tools	SparkleBox	Art	$97.13	3.5	Yes	SKU-ART-0005	5

Table 3. First five records of the scraped DataFrame (as displayed by df.head())

The missing-value count of zero for the Image column may seem surprising because the column holds no visible text. The reason is that the scraped Image cell is an empty string, not a null value, so isnull() does not count it. When the CSV file is reloaded in Part B, the empty strings are read as NaN, as can be seen in the Chapter 8 output of the loading step. This column is not used in any calculation.

CHAPTER 3
DATA COLLECTION AND PREPROCESSING

3.1 Overview of Data Preprocessing

Data collected from a web page is text. Before it can be analysed, it must be collected completely, arranged in a table, checked for problems and converted to proper data types. This chapter explains the code of Part A, which performs these tasks in ten notebook cells: installation (Cells 1 and 2), browser test (Cell 3), scraping (Cell 4), DataFrame creation (Cell 5), validation (Cell 6), cleaning (Cell 7), data-type verification (Cell 8), saving (Cell 9) and downloading (Cell 10). The complete code is given in the Appendix; only the important parts are reproduced here.

3.2 Installing Required Libraries

Purpose

To install the Python packages that the scraping and saving steps need.

```
!pip install playwright openpyxl
!playwright install chromium
```

The first line, `pip install playwright openpyxl`, installs Playwright, which automates the browser, and OpenPyXL, which Pandas uses as the engine for writing .xlsx files. The second line, `playwright install chromium`, downloads the Chromium browser that Playwright controls. The leading exclamation mark is Google Colab (Jupyter) notation for running a shell command from a notebook cell. Without these installations the later cells could neither scrape the website nor write the Excel file.

3.3 Installing Chromium

Purpose

To make sure that Chromium and the system libraries it needs are available in the Colab environment.

```
!python -m playwright install --with-deps chromium
```

The `--with-deps` option installs the operating-system dependencies that Chromium requires in addition to the browser itself. A browser that is installed without its dependencies may fail to start in a fresh Colab session, so this cell is run before the browser is used.

3.4 Testing the Browser

Purpose

To confirm that Playwright can start and close Chromium before running the longer scraping code.

```
from playwright.async_api import async_playwright

async def test_browser():

    async with async_playwright() as p:

        browser = await p.chromium.launch(
            headless=True
        )

        print("Browser started successfully!")

        await browser.close()

await test_browser()
```

`async_playwright()` starts Playwright using its asynchronous interface, which is the interface that works inside a Colab notebook. `p.chromium.launch(headless=True)` starts Chromium without a visible window. The program prints "Browser started successfully!" and then closes the browser with `browser.close()`. The keywords `async` and `await` are used because the asynchronous API performs browser operations as awaitable calls. The expected output is the single line "Browser started successfully!" (see Chapter 8). This test is useful because an installation problem can be found quickly, before the 20-page scraping loop begins.

3.5 Opening the Target Website

Purpose

To start the scraping function, create the result list and open the Data Table page.

```
async def scrape_products():

    url = "https://scrapingsandbox.com/data-table"

    all_products = []

    async with async_playwright() as p:

        browser = await p.chromium.launch(headless=True)

        page = await browser.new_page()

        await page.goto(url, wait_until="domcontentloaded")

        await page.locator("table").wait_for()

        print("Website loaded successfully!")
```

The URL of the Data Table page is stored in `url`, and an empty list, `all_products`, is created to hold the scraped rows. `p.chromium.launch(headless=True)` starts the browser and `browser.new_page()` opens a new tab. `page.goto(url, wait_until="domcontentloaded")` opens the website and continues once the basic page content has loaded. Because the table may still be drawn after that moment, `page.locator("table").wait_for()` pauses the program until the table element is available. Only then does the code print "Website loaded successfully!". Waiting for the table prevents the scraper from reading an empty page.

3.6 Pagination and Product Collection

Purpose

To visit all 20 pages of the table.

```
# There are 20 pages
for page_number in range(1, 21):

    print("Scraping page:", page_number)

    # Click page number
    if page_number > 1:

        button = page.get_by_role(
            "button",
            name=str(page_number),
            exact=True
        )

        await button.click()

        # Wait for table to update
        await page.wait_for_timeout(500)
```

`range(1, 21)` produces the page numbers 1 to 20. Page 1 is already displayed after the website loads, so the click is needed only when `page_number > 1`. For the later pages, `page.get_by_role("button", name=str(page_number), exact=True)` finds the pagination button whose visible name is exactly the page number, and `button.click()` clicks it. The call `page.wait_for_timeout(500)` waits 500 milliseconds so that the table has time to show the new rows before they are read. Using `exact=True` is important: without it, searching for the name "1" could also match buttons such as "10" or "11".

3.7 Extracting Table Rows

Purpose

To find all product rows on the current page.

```
# Get table rows
rows = page.locator("table tbody tr")

row_count = await rows.count()

print("Rows found:", row_count)
```

`page.locator("table tbody tr")` selects every row inside the table body (`tbody`), so the header row is not included. `await rows.count()` returns the number of rows on the page, which is printed as "Rows found". Since each page shows 25 products, the expected value is 25 on every page; this printed value is a quick check that each page has been loaded completely.

3.8 Extracting Table Cells

Purpose

To read the text of every cell in every row and store a row as a list of values.

```
# Extract every row
for i in range(row_count):

    cells = rows.nth(i).locator("td")

    cell_count = await cells.count()

    values = []

    for j in range(cell_count):

        value = await cells.nth(j).inner_text()

        values.append(value.strip())

    if len(values) >= 10:

        all_products.append(values)
```

The outer loop `for i in range(row_count)` processes one row at a time; `rows.nth(i)` selects the i-th row and `.locator("td")` selects the cells (`td` elements) of that row. The inner loop visits each cell. `inner_text()` returns the visible text of the cell and `strip()` removes leading and trailing whitespace. The cleaned value is added to the list `values`. The condition `if len(values) >= 10` accepts the row only when it contains at least the ten expected cells, which protects the dataset from incomplete rows. Valid rows are appended to `all_products`, so after all 20 pages `all_products` is a list of lists. After the loop, `browser.close()` closes Chromium and the function returns the list.

```
        await browser.close()

    return all_products

all_products = await scrape_products()

print("\n---------------------------------")
print("TOTAL PRODUCTS SCRAPED:", len(all_products))
```

Running `await scrape_products()` executes the function and prints the total number of scraped products. The expected value is 500.

3.9 Creating the Pandas DataFrame

Purpose

To convert the list of scraped rows into a structured table.

```
columns = [
    "ID",
    "Image",
    "Product_Name",
    "Vendor",
    "Category",
    "Price",
    "Rating",
    "Stock",
    "SKU",
    "Variants"
]

df = pd.DataFrame(all_products, columns=columns)

print("Dataset shape:", df.shape)

df.head()
```

`columns` lists the ten column names in the same order as the cells on the page. `pd.DataFrame(all_products, columns=columns)` converts the list of lists into a DataFrame in which every inner list becomes a row. `df.shape` gives the number of rows and columns, and `df.head()` displays the first five rows. A DataFrame is needed because Pandas provides the cleaning, grouping and statistical functions used in the rest of the project.

3.10 Dataset Validation

Purpose

To check that the scraped dataset is complete before it is cleaned.

```
print("Number of rows:", len(df))
print("Number of columns:", len(df.columns))

print("\nColumn names:")
print(df.columns.tolist())

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:")
print(df.duplicated().sum())
```

- `len(df)` and `len(df.columns)` give the number of rows and columns.
- `df.columns.tolist()` lists the column names so that they can be verified.
- `df.isnull().sum()` counts the missing (null) values in each column.
- `df.duplicated().sum()` counts rows that are exact duplicates of an earlier row.

The supplied output shows 500 rows, 10 columns, zero missing values in every column and zero duplicate rows (Chapter 8). Validation is necessary because errors in the data, such as incomplete or repeated rows, would otherwise silently affect every later statistic.

3.11 Handling Missing Values

Missing values are handled in two ways in the implementation. First, they are detected with `isnull().sum()` during validation, and the output reported zero missing values in the scraped DataFrame. Second, the numeric conversion of Rating and Variants uses `errors="coerce"`, so that any value that cannot be converted becomes NaN instead of stopping the program (Section 3.14). The code does not delete or fill missing values, because none were reported. Where the analysis calculates statistics, Pandas methods such as mean(), min(), max(), median() and corr() skip missing values automatically.

3.12 Checking Duplicate Records

`df.duplicated()` marks a row as a duplicate when all its column values are identical to those of an earlier row, and `.sum()` counts the marked rows. The supplied output shows 0 duplicate rows. In addition, every product has a different ID and a different SKU in the displayed records, so no row removal was required. No `drop_duplicates()` call is present in the code, and none was needed.

3.13 Cleaning the Price Column

Purpose

To convert prices such as "$1,299.50" or "$155.62" from text into numbers.

```
df["Price"] = (
    df["Price"]
    .str.replace("$", "", regex=False)
```

The text values contain a dollar symbol, and possibly a comma used as a thousands separator, which prevent direct conversion. `.str.replace("$", "", regex=False)` removes the dollar symbol and `.str.replace(",", "", regex=False)` removes commas; `regex=False` makes Pandas treat the symbol literally, because "$" has a special meaning in regular expressions. `.astype(float)` then converts the remaining text, for example "155.62", to the floating-point number 155.62. Method chaining places the three operations in one readable statement. A numeric Price is essential for the average, minimum, maximum, median, grouping, sorting, classification, correlation and every chart.

3.14 Converting Rating and Variants

```
df["Rating"] = pd.to_numeric(
    df["Rating"],
    errors="coerce"
)

df["Variants"] = pd.to_numeric(
    df["Variants"],
    errors="coerce"
)
```

`pd.to_numeric()` converts text to numbers. The argument `errors="coerce"` means that a value that cannot be converted is replaced by NaN instead of causing an error. This is a safe choice for scraped data, in which an unexpected entry in a cell is possible. After conversion, Rating and Variants can be used in calculations such as the price–rating correlation.

3.15 Verifying Data Types

```
print(df.dtypes)
```

`df.dtypes` displays the data type of every column. After the cleaning step, Price is expected to be float64 and Rating and Variants numeric, while the other columns remain object (text). This check confirms that the conversions of Section 3.13 and 3.14 were applied before the dataset is saved.

3.16 Saving the Cleaned Dataset

```
df.to_csv("scraped_product_data.csv", index=False)

df.to_excel("scraped_product_data.xlsx", index=False)

print("Dataset saved successfully!")
```

```
from google.colab import files

files.download("scraped_product_data.csv")
```

`df.to_csv("scraped_product_data.csv", index=False)` writes the cleaned DataFrame to a CSV file and `df.to_excel("scraped_product_data.xlsx",`

`index=False)` writes the same data to an Excel file (using OpenPyXL). `index=False` prevents Pandas from writing its own row numbers as an extra column. The CSV file is the input of Part B. Finally, `files.download("scraped_product_data.csv")` from `google.colab` downloads the CSV file to the user's computer, because files saved in a Colab session are temporary. The output "Dataset saved successfully!" is shown in Chapter 8.

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

4.1 Proposed Methodology

The proposed methodology converts the manual task of collecting and comparing product data into an automated pipeline with the following stages:

1. **Data collection** – Playwright opens the Scraping Sandbox Data Table in headless Chromium and visits 20 pages.
2. **Data structuring** – the scraped rows are stored in a list and converted into a Pandas DataFrame with ten columns.
3. **Data validation** – row count, column names, missing values and duplicate rows are checked.
4. **Data cleaning** – Price is converted to float; Rating and Variants are converted to numeric values.
5. **Data saving** – the cleaned dataset is saved as CSV and Excel files.
6. **Exploratory analysis** – the CSV file is reloaded and price statistics, category, vendor and price-range comparisons and the price–rating correlation are calculated.
7. **Visualisation** – five Matplotlib charts present the distribution and comparisons.
8. **Export of results** – the summary and analysis tables are saved as CSV files.

4.2 System Architecture

The system can be understood as six layers. Each layer receives the output of the layer above it and passes its own output to the layer below. The layers are shown in Figure 1: the data source, the collection layer built on Playwright and Chromium, the preparation layer in which the DataFrame is validated and cleaned, the analysis layer, the visualisation layer built on Matplotlib, and the storage layer that writes CSV and Excel files.

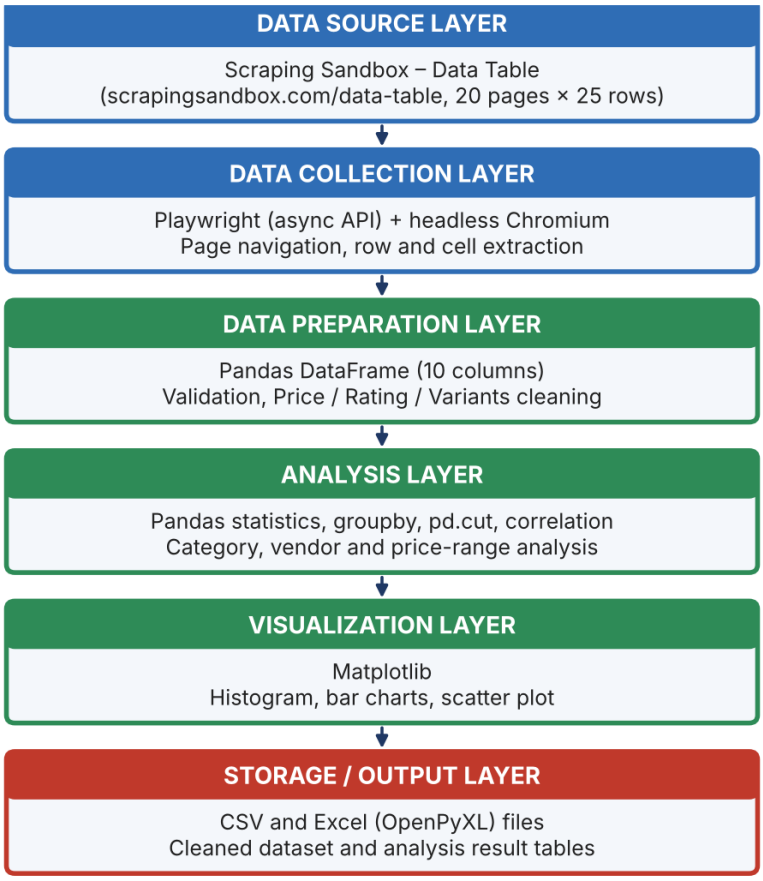


Figure 1. System architecture of the Web Scraping and Product Price Analysis System

4.3 Project Workflow

The workflow of the project follows the sequence shown in Figure 2. The first four steps (blue) belong to data collection, the next six steps (green) to processing and analysis, and the last step (red) to the saved results.

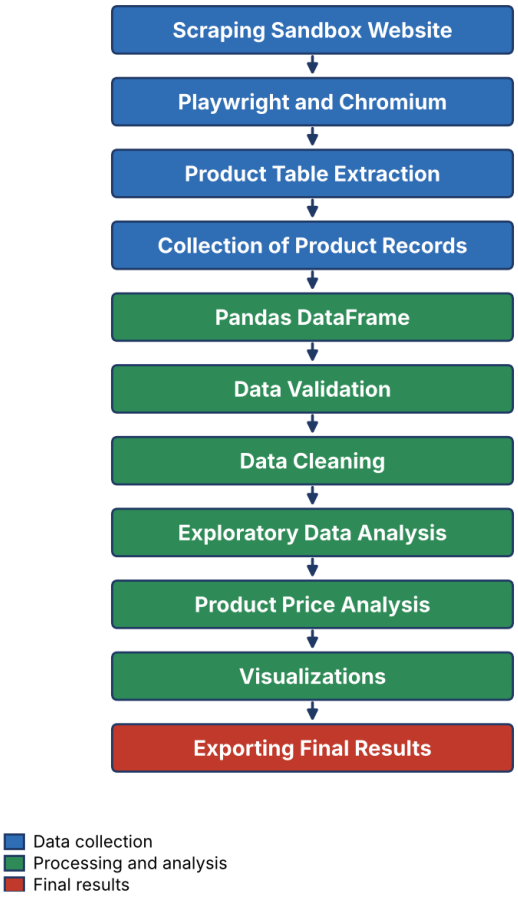


Figure 2. Project workflow from the Scraping Sandbox website to the exported results

4.4 Flowchart

Figure 3 shows the control flow of the implemented code. The loop that visits the pages is the most important part: for each page number from 1 to 20 the program clicks the page button when the page number is greater than 1, reads the rows and cells, and appends each row that contains at least ten values to the result list. When all pages have been read, the browser is closed and the processing and analysis stages follow.

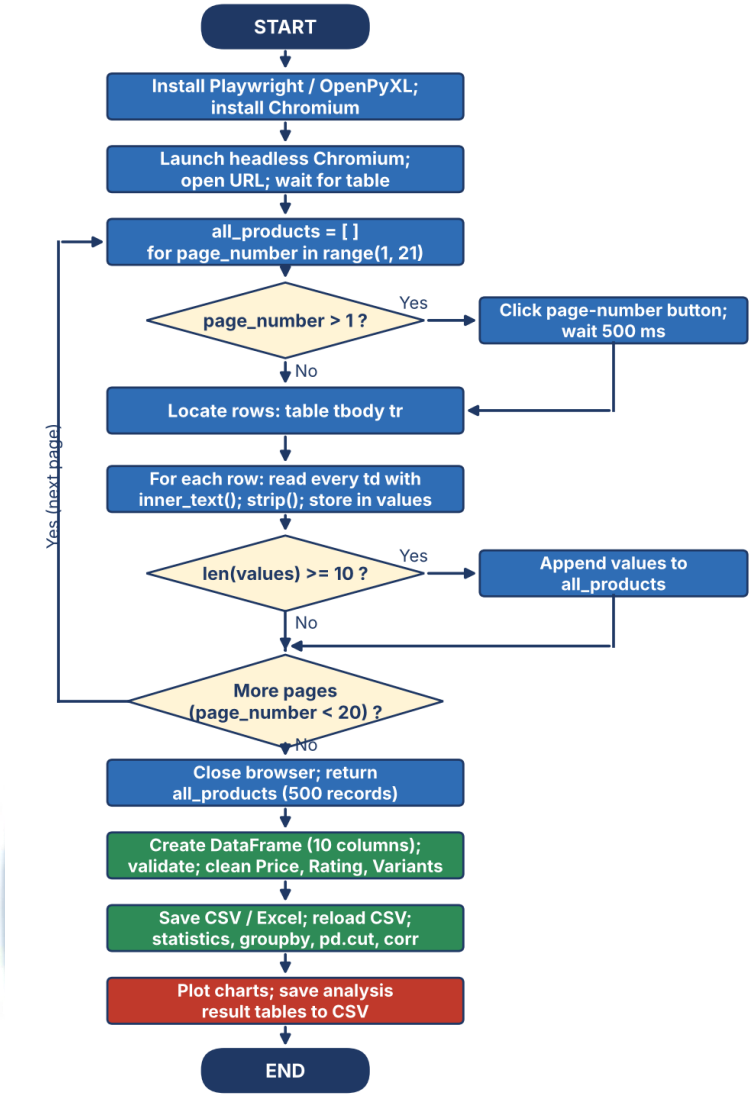


Figure 3. Flowchart of the scraping, preprocessing and analysis process

4.5 Tools and Technologies

Technology	Type	Role in the project
Python	Programming language	Language used for the complete project.
Google Colab	Cloud notebook environment	Runs the code in notebook cells and provides files.download().
Playwright	Browser-automation library	Controls Chromium to open the site, click pagination buttons and read the table.
Chromium	Web browser	Headless browser that renders the Data Table page.
Pandas	Data-analysis library	DataFrame creation, cleaning, grouping, statistics, classification and correlation.
Matplotlib	Visualisation library	Histogram, bar charts and scatter plot.
OpenPyXL	Spreadsheet library	Enables Pandas to write the .xlsx file.
CSV / Excel files	Data formats	Store the cleaned dataset and the analysis results.

Table 4. Tools and technologies used in the project

Two general approaches exist for collecting data from web pages. Table 5 compares them to explain why browser automation was used. The comparison is general in nature; the project itself implements only the browser-automation approach.

Aspect	Reading the page source directly	Browser automation (used here)
How it works	Downloads the page source and searches it for the required elements.	Opens the page in a real browser and reads what the browser displays.
Pages changed by buttons	Cannot click buttons by itself.	Can click pagination buttons and wait for the table to update.
Waiting for content	Not applicable.	Can wait until an element, such as the table, is available.
Resources needed	Light.	Heavier, because a browser is started.

Table 5. Approaches to collecting data from web pages

4.6 Algorithms and Functions Used

Function / method	Library	Purpose
`async_playwright(), chromium.launch(headless=True)`	Playwright	Start Playwright and launch headless Chromium.
`page.goto(), locator().wait_for()`	Playwright	Open the URL and wait for the table.
`get_by_role("button", name=..., exact=True), click()`	Playwright	Select and click the page-number button.
`locator("table tbody tr"), locator("td"), count(), nth()`	Playwright	Find and iterate over rows and cells.
`inner_text(), strip()`	Playwright / Python	Read cell text and remove extra spaces.
`pd.DataFrame()`	Pandas	Create the dataset from the scraped rows.
`isnull().sum(), duplicated().sum()`	Pandas	Validate missing values and duplicates.

Function / method	Library	Purpose
`str.replace(), astype(float), pd.to_numeric()`	Pandas	Clean Price, Rating and Variants.
`to_csv(), to_excel(), read_csv()`	Pandas	Save and reload the dataset.
`mean(), min(), max(), median(), describe()`	Pandas	Price statistics.
`idxmin(), idxmax(), loc[]`	Pandas	Find and display the cheapest and most expensive product.
`groupby(), agg(), sort_values(), head()`	Pandas	Category and vendor analysis; top-10 lists.
`pd.cut(), value_counts()`	Pandas	Price-range classification and counting.
`corr()`	Pandas	Price–rating correlation.
`hist(), plot(kind="bar"), scatter()`	Matplotlib	Draw the charts.

Table 6. Main functions used in the implementation

4.7 Functional Modules

Module 1: Browser setup

Installs Playwright, OpenPyXL and Chromium and tests that headless Chromium starts (Cells 1–3). It prepares the environment so that the scraper can run.

Module 2: Web scraping

The function scrape_products() opens the Data Table, loops over 20 pages, clicks the page buttons, reads rows and cells and returns the list of records (Cell 4).

Module 3: Dataset creation

Converts the list of records into a DataFrame with ten named columns (Cell 5).

Module 4: Dataset validation

Checks rows, columns, column names, missing values and duplicate rows (Cell 6).

Module 5: Data cleaning

Converts Price to float and Rating and Variants to numbers; verifies the data types; saves the dataset (Cells 7–9).

Module 6: Statistical analysis

Calculates the average, minimum, maximum and median price and the describe() summary; finds the cheapest and most expensive product (Part B, steps 3–6).

Module 7: Category analysis

Groups the products by Category to calculate the average price and the minimum, maximum and mean price (steps 8–10).

Module 8: Price-range classification

Divides prices into three equal-width intervals labelled Low Price, Medium Price and High Price and counts the products in each (steps 13–14).

Module 9: Vendor analysis

Groups the products by Vendor to calculate and sort the average price (steps 17–18).

Module 10: Price–rating correlation

Draws the price–rating scatter plot and calculates the correlation coefficient (steps 15–16).

Module 11: Visualisation

Produces the histogram, category chart, price-range chart, scatter plot and vendor chart with Matplotlib (steps 7, 9, 14, 15, 18).

Module 12: Exporting results

Builds the final summary table and saves the summary, category, top-10 expensive and top-10 cheapest tables as CSV files (steps 19–20).

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment

The project was developed and executed in Google Colab, a cloud-based notebook environment in which code is organised in cells. Part A uses ten cells and Part B uses twenty numbered analysis steps. The language is Python. The scraping code uses top-level `await` inside notebook cells, which Colab supports, so it is run as notebook cells and not as a standalone script. The numerical results shown in this report were produced by the project's own execution and are taken from the notebook output screenshots.

5.2 Libraries Used

Library	Import statement	Used for
Playwright	`from playwright.async_api import async_playwright`	Automating Chromium.
Pandas	`import pandas as pd`	DataFrames, cleaning, statistics, grouping.
Matplotlib	`import matplotlib.pyplot as plt`	Charts.
OpenPyXL	`(installed with pip; used internally by Pandas)`	Writing .xlsx files.
google.colab	`from google.colab import files`	Downloading the CSV file from Colab.

Table 7. Libraries imported by the code

5.3 Web Scraping Implementation

The web scraping implementation is the function `scrape_products()` in Cell 4. It has been explained line by line in Sections 3.5 to 3.8. In summary, it opens the page, waits for the table, loops through the 20 pages, clicks the page buttons, extracts the rows and cells with `inner_text()` and `strip()`, keeps rows with at least ten values and returns 500 records. Its complete listing is in Part A of the Appendix.

5.4 DataFrame Creation

The DataFrame is created in Cell 5 with `pd.DataFrame(all_products, columns=columns)` (Section 3.9). The resulting shape is (500, 10).

5.5 Data Validation Implementation

Validation is implemented in Cell 6 (Section 3.10) with `len()`, `columns.tolist()`, `isnull().sum()` and `duplicated().sum()`.

5.6 Data Cleaning Implementation

Cleaning is implemented in Cell 7 (Sections 3.13 and 3.14): the Price column is converted to float and the Rating and Variants columns are converted with `pd.to_numeric(..., errors="coerce")`. The data types are verified in Cell 8.

5.7 Dataset Saving and Download

Cell 9 saves `scraped_product_data.csv` and `scraped_product_data.xlsx`, and Cell 10 downloads the CSV file (Section 3.16).

5.8 Loading the Dataset for Analysis

Purpose

To start Part B by loading the saved CSV file into a new DataFrame.

```
import pandas as pd

df = pd.read_csv("scraped_product_data.csv")

print("Dataset loaded successfully!")
print("Number of products:", len(df))
print("Number of columns:", len(df.columns))

df.head()
```

`pd.read_csv("scraped_product_data.csv")` reads the CSV file into the DataFrame `df`. The program prints the number of products with `len(df)` and the number of columns with `len(df.columns)`, and `df.head()` shows the first five rows.

Part B reloads the file instead of continuing with the in-memory DataFrame so that the analysis can be run separately, at any later time, from the saved dataset.

Expected output

The notebook output shows 500 products and 10 columns, followed by the first five rows (Figure 4). In the reloaded data the Image column appears as NaN because its values are empty.

Why this step is needed

Loading from the saved file makes Part B independent of the scraping session and shows that the saved CSV file is complete.

```
Dataset loaded successfully!
Number of products: 500
Number of columns: 10
```

	ID	Image	Product_Name	Vendor	Category	Price	Rating	Stock	SKU	Variants
0	1	NaN	Lightweight Probiotics	SoundWave	Health	155.62	4.0	Yes	SKU-HEA-0001	4
1	2	NaN	Wireless LED Desk Lamp	SoundWave	Electronics	145.40	4.6	No	SKU-ELE-0002	2
2	3	NaN	Deluxe Tea Collection	StyleCraft	Food & Beverage	185.01	3.5	Yes	SKU-FOO-0003	3
3	4	NaN	Essential Tire Gauge	TechVault	Automotive	70.86	3.6	No	SKU-AUT-0004	5
4	5	NaN	Smart Carving Tools	SparkleBox	Art	97.13	3.5	Yes	SKU-ART-0005	5

Figure 4. Output of the dataset-loading step: number of products, number of columns and df.head()

5.9 Price Column Preparation

Purpose

To make sure that Price is numeric after reloading the CSV file.

```
df["Price"] = (
    df["Price"]
    .astype(str)
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
)

df["Price"] = pd.to_numeric(df["Price"], errors="coerce")

print(df["Price"].head())
```

`astype(str)` ensures that text methods can be applied to every value. `str.replace("$", "", regex=False)` and `str.replace(",", "", regex=False)` remove the dollar symbol and commas if they are present. `pd.to_numeric(df["Price"], errors="coerce")` converts the result to numbers, turning any invalid entry into NaN. Because Price was already converted in Part A, this step is a safety measure that makes Part B correct even if it is run on a CSV file that still contains currency symbols.

Expected output

`df["Price"].head()` prints the first five numeric prices.

Why this step is needed

Every statistic, sort and chart that follows needs Price to be a number.

5.10 Basic Price Analysis

Purpose

To calculate the main summary values of the Price column.

```
print("========== PRODUCT PRICE ANALYSIS ==========")

print("Average Price   :", round(df["Price"].mean(), 2))
print("Minimum Price   :", round(df["Price"].min(), 2))
print("Maximum Price   :", round(df["Price"].max(), 2))
print("Median Price    :", round(df["Price"].median(), 2))
```

`mean()` returns the arithmetic average, `min()` the smallest value, `max()` the largest value and `median()` the middle value of the sorted prices. `round(..., 2)` limits each result to two decimal places. The median is useful beside the mean because it is not affected as strongly by unusually high or low prices.

Why this step is needed

These four values give a first picture of the price level and range.

5.11 Complete Price Statistics

Purpose

To obtain a full statistical summary of Price in one call.

```
price_statistics = df["Price"].describe()

print("========== PRICE STATISTICS ==========")
print(price_statistics)
```

`describe()` returns the count of values, mean, standard deviation (std), minimum, the 25%, 50% and 75% quartiles and the maximum. The 50% value is the median. The standard deviation measures how widely prices are spread around the mean, and the quartiles show the values below which 25%, 50% and 75% of the prices lie.

Why this step is needed

It shows the spread and shape of the price values, not just a single average.

5.12 Cheapest Product Identification

Purpose

To find the complete record of the lowest-priced product.

```
cheapest_product = df.loc[df["Price"].idxmin()]

print("========== CHEAPEST PRODUCT ==========")
print("Product Name :", cheapest_product["Product_Name"])
print("Category     :", cheapest_product["Category"])
print("Vendor       :", cheapest_product["Vendor"])
print("Price        :", cheapest_product["Price"])
print("Rating       :", cheapest_product["Rating"])
```

`df["Price"].idxmin()` returns the index label of the row with the smallest price. `df.loc[...]` retrieves the full row for that label, and the print statements display its Product_Name, Category, Vendor, Price and Rating.

Why this step is needed

It identifies which product is at the bottom of the price range and gives its details.

5.13 Most Expensive Product Identification

Purpose

To find the complete record of the highest-priced product.

```
expensive_product = df.loc[df["Price"].idxmax()]

print("========== MOST EXPENSIVE PRODUCT ==========")
print("Product Name :", expensive_product["Product_Name"])
print("Category     :", expensive_product["Category"])
print("Vendor       :", expensive_product["Vendor"])
print("Price        :", expensive_product["Price"])
print("Rating       :", expensive_product["Rating"])
```

The logic is identical to the previous step, but `idxmax()` returns the index of the largest price.

Why this step is needed

It identifies the top of the price range.

5.14 Category-wise Price Analysis

Purpose

To compare prices across product categories.

```
category_price = (
    df.groupby("Category")["Price"]
      .mean()
      .sort_values(ascending=False)
)

print("========== AVERAGE PRICE BY CATEGORY ==========")
print(category_price)

category_price_range = df.groupby("Category")["Price"].agg(
    ["min", "max", "mean"]
)

category_price_range = category_price_range.round(2)

print("========== CATEGORY PRICE RANGE ==========")
print(category_price_range)
```

`df.groupby("Category")["Price"]` splits the products into groups by category and selects the Price column of each group. `.mean()` calculates the average of each group and `.sort_values(ascending=False)` orders the categories from the highest to the lowest average. The second code block uses `.agg(["min", "max", "mean"])` to calculate three statistics for each category at once; `round(2)` rounds the table to two decimals. The first block is also used for the bar chart in Section 6.3 and the second block is saved in `category_price_analysis.csv`.

Why this step is needed

Category-wise results show whether some categories are priced higher or lower than others.

5.15 Top 10 Expensive Products

Purpose

To list the ten highest-priced products.

```
top_expensive = (
    df.sort_values("Price", ascending=False)
      [["Product_Name", "Category", "Vendor", "Price", "Rating"]]
      .head(10)
)

print("========== TOP 10 MOST EXPENSIVE PRODUCTS ==========")

top_expensive
```

`sort_values("Price", ascending=False)` sorts the products from the highest to the lowest price. The double square brackets select the columns Product_Name, Category, Vendor, Price and Rating, and `head(10)` keeps the first ten rows. The result is stored in `top_expensive` for later saving.

Why this step is needed

It shows which individual products define the upper end of the price range.

5.16 Top 10 Cheapest Products

Purpose

To list the ten lowest-priced products.

```
top_cheapest = (
    df.sort_values("Price", ascending=True)
      [["Product_Name", "Category", "Vendor", "Price", "Rating"]]
      .head(10)
)

print("========== TOP 10 CHEAPEST PRODUCTS ==========")

top_cheapest
```

The code is the same as above with `ascending=True`, so the lowest prices come first. The result is stored in `top_cheapest`.

Why this step is needed

It shows the products at the lower end of the price range.

5.17 Price Range Classification

Purpose

To divide the products into Low Price, Medium Price and High Price groups.

```
min_price = df["Price"].min()
max_price = df["Price"].max()

df["Price_Range"] = pd.cut(
    df["Price"],
    bins=3,
    labels=["Low Price", "Medium Price", "High Price"]
)

print("========== PRICE RANGE ANALYSIS ==========")

print(df["Price_Range"].value_counts())
```

`pd.cut(df["Price"], bins=3, labels=[...])` divides the range of prices into **three equal-width intervals** and assigns each product the label of its interval. A new column, `Price_Range`, stores the label. `value_counts()` counts how many products fall into each label. The variables `min_price` and `max_price` are calculated in the code but are not used by `pd.cut()`, which determines the interval limits from the data by itself.

Because the intervals have equal width, the groups are not guaranteed to contain equal numbers of products. The group sizes depend on how the prices are distributed.

Why this step is needed

Simple classes make it easy to see how many products are at low, medium and high price levels.

The `Price_Range` classification is based only on the Price column. The labels "Low", "Medium" and "High" are relative to the prices in this dataset; they are not general market categories.

5.18 Price-Rating Relationship

Purpose

To inspect visually whether price and rating appear to be related.

```
plt.figure(figsize=(10, 6))

plt.scatter(
    df["Price"],
    df["Rating"],
    alpha=0.6
)

plt.title("Product Price vs Rating")
plt.xlabel("Price")
plt.ylabel("Rating")

plt.tight_layout()
plt.show()
```

`plt.scatter(df["Price"], df["Rating"], alpha=0.6)` draws one point for every product, with Price on the x-axis and Rating on the y-axis. `alpha=0.6` makes the points semi-transparent so that overlapping points remain visible.

Why this step is needed

A scatter plot reveals patterns such as a rising or falling trend, or the absence of any trend, that a single number can hide.

5.19 Price-Rating Correlation

Purpose

To measure the strength and direction of the linear relationship between price and rating.

```
price_rating_correlation = df["Price"].corr(df["Rating"])

print(
    "Correlation between Price and Rating:",
    round(price_rating_correlation, 3)
)
```

`df["Price"].corr(df["Rating"])` calculates the Pearson correlation coefficient, which lies between −1 and +1. A value near +1 indicates that high prices tend to go with high ratings, a value near −1 indicates the opposite, and a value near 0 indicates little linear relationship. The result is rounded to three decimals.

Correlation does not prove causation. Even a large correlation would only show that two variables move together, not that one causes the other.

Why this step is needed

It gives a numerical answer to the question that the scatter plot raises visually.

5.20 Vendor-wise Average Price

Purpose

To compare the average price of products across vendors.

```
vendor_price = (
    df.groupby("Vendor")["Price"]
      .mean()
      .sort_values(ascending=False)
)

print("========== AVERAGE PRICE BY VENDOR ==========")
print(vendor_price)
```

`groupby("Vendor")["Price"].mean()` calculates the average price for each vendor and `sort_values(ascending=False)` ranks the vendors from the highest to the lowest average. The result is also used for the vendor bar chart (Section 6.6).

Why this step is needed

It shows whether some vendors sell products at higher average prices than others.

5.21 Final Summary Table

Purpose

To collect the key results in one table.

```
price_summary = pd.DataFrame({
    "Metric": [
        "Total Products",
        "Average Price",
        "Minimum Price",
        "Maximum Price",
        "Median Price",
        "Price-Rating Correlation"
    ],

    "Value": [
        len(df),
        round(df["Price"].mean(), 2),
        round(df["Price"].min(), 2),
        round(df["Price"].max(), 2),
        round(df["Price"].median(), 2),
        round(df["Price"].corr(df["Rating"]), 3)
    ]
})

price_summary
```

A DataFrame with the columns Metric and Value is created. The metrics are **Total Products** (`len(df)`), **Average Price** (mean), **Minimum Price** (min), **Maximum Price** (max), **Median Price** (median) and **Price-Rating Correlation** (`corr()`). The values are rounded to two decimals, and the correlation to three decimals. The numeric values are shown in the notebook as a single column, so the Total Products value appears as 500.00.

Why this step is needed

A single summary table is easy to present, save and include in a report.

5.22 Saving Analysis Results

Purpose

To save the analysis tables for later use.

```
price_summary.to_csv(
    "product_price_analysis_summary.csv",
    index=False
)

category_price_range.to_csv(
    "category_price_analysis.csv"
)

top_expensive.to_csv(
    "top_10_expensive_products.csv",
    index=False
)

top_cheapest.to_csv(
    "top_10_cheapest_products.csv",
    index=False
)

print("Price analysis files saved successfully!")
```

`to_csv()` writes a DataFrame to a CSV file. Four files are produced: `product_price_analysis_summary.csv`, `category_price_analysis.csv`, `top_10_expensive_products.csv` and `top_10_cheapest_products.csv`. `index=False` is used for the three tables whose index is only a row number; for the category table the index (the category names) is kept because it carries the category labels.

Expected output

The message "Price analysis files saved successfully!" is printed.

Why this step is needed

Saved files allow the results to be reused, shared or documented without running the analysis again.

CHAPTER 6
DATA VISUALIZATION AND RESULTS

6.1 Introduction to Data Visualisation

Data visualisation presents numerical results as pictures so that patterns can be seen quickly. The project uses Matplotlib to draw five charts: a histogram, three bar charts and a scatter plot. All five charts shown in this chapter are the actual outputs generated by the project's code and were taken from the notebook output. Each chart is explained below in terms of what the code draws; observations are limited to what can be read from the charts.

6.2 Product Price Distribution Histogram

```
plt.figure(figsize=(10, 6))

plt.hist(df["Price"], bins=20)

plt.title("Product Price Distribution")
plt.xlabel("Price")
plt.ylabel("Number of Products")

plt.tight_layout()
plt.show()
```

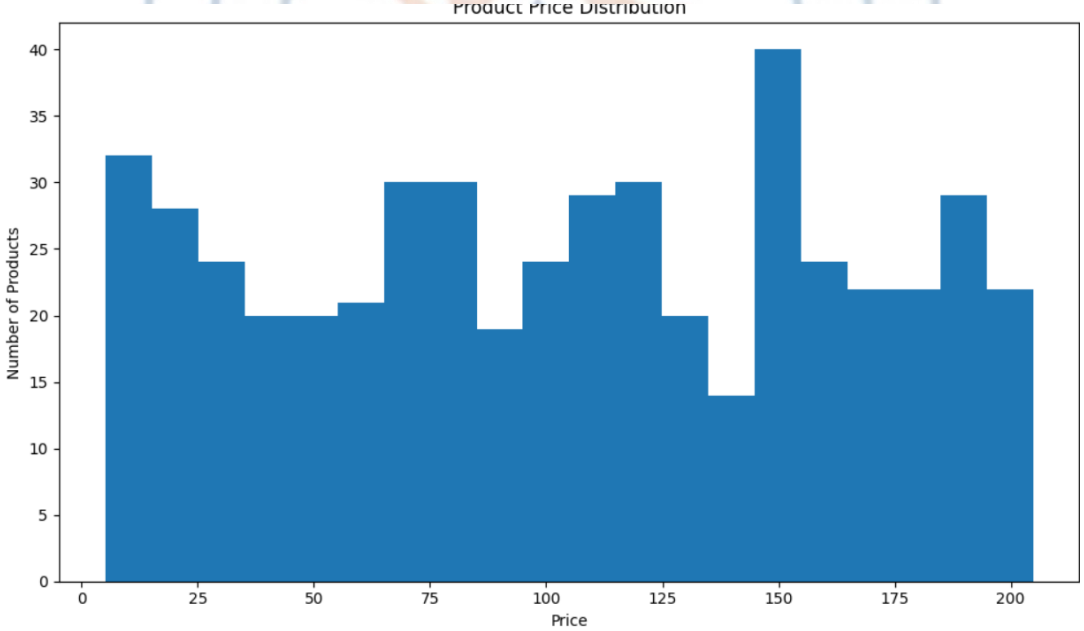


Figure 5. Product Price Distribution histogram

The x-axis shows Price and the y-axis shows the Number of Products. `plt.hist(df["Price"], bins=20)` divides the range of prices into 20 equal intervals; each bar represents the number of products whose price falls inside one interval, and the height of the bar is that count. The histogram therefore shows how the prices are **distributed** across the price range; it does not show the average price. In this chart the bars are spread over the whole range from the lowest to the highest price and no single sharp peak dominates.

The bar heights range between roughly 14 and 40 products, with the tallest bar near a price of 150.

6.3 Average Price by Category Bar Chart

```
plt.figure(figsize=(12, 6))

category_price.plot(kind="bar")

plt.title("Average Product Price by Category")
plt.xlabel("Category")
plt.ylabel("Average Price")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

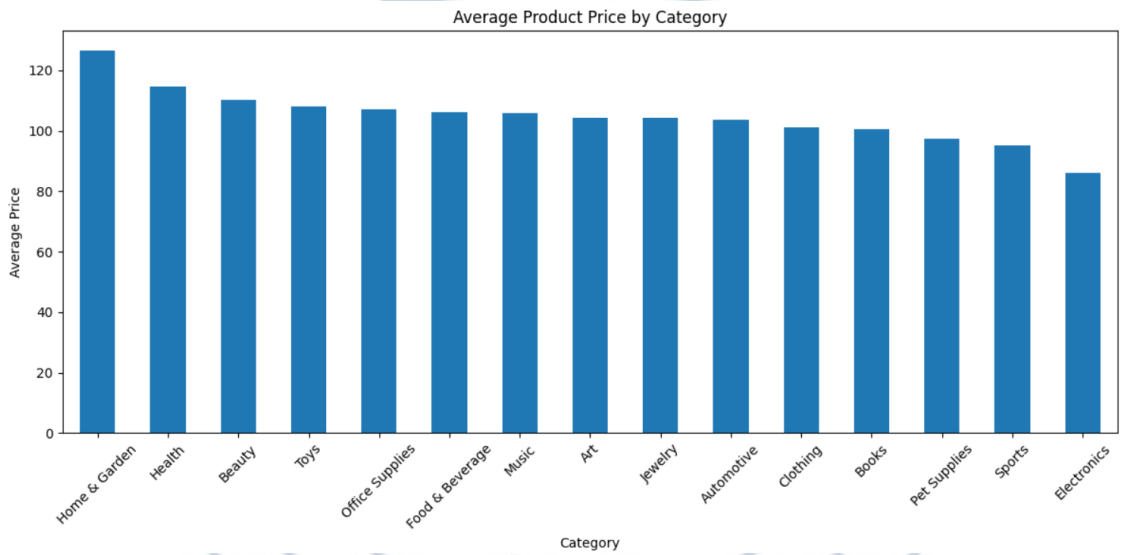


Figure 6. Average Product Price by Category

Each bar represents one category and its height is the average price of the products in that category. Because the data was sorted with `sort_values(ascending=False)` before plotting, the bars run from the highest average on the left to the lowest on the right. The chart makes it easy to compare the average price level of the categories.
`plt.xticks(rotation=45)` rotates the category names so that they remain readable. The values behind the bars are tabulated in Section 7.4.

6.4 Product Price Range Chart

```
price_range_count = df["Price_Range"].value_counts()

plt.figure(figsize=(8, 5))

price_range_count.plot(kind="bar")

plt.title("Products by Price Range")
plt.xlabel("Price Range")
plt.ylabel("Number of Products")

plt.tight_layout()
plt.show()
```

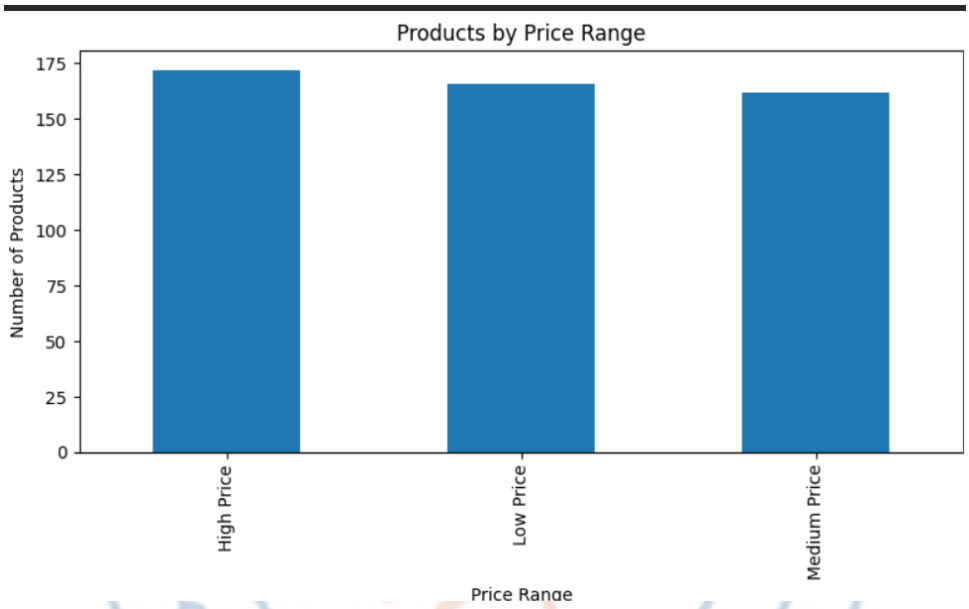


Figure 7. Products by Price Range

The chart shows the number of products assigned to each of the three groups produced by `pd.cut()`: Low Price, Medium Price and High Price. The bars appear in the order High Price, Low Price, Medium Price because `value_counts()` sorts the groups by count, from the largest to the smallest, and the bar chart follows this order. The three bars have heights that are close to each other, so the products are spread over the three price intervals in similar numbers. The exact counts are printed by the `value_counts()` output of Section 5.17.

6.5 Product Price vs Rating Scatter Plot

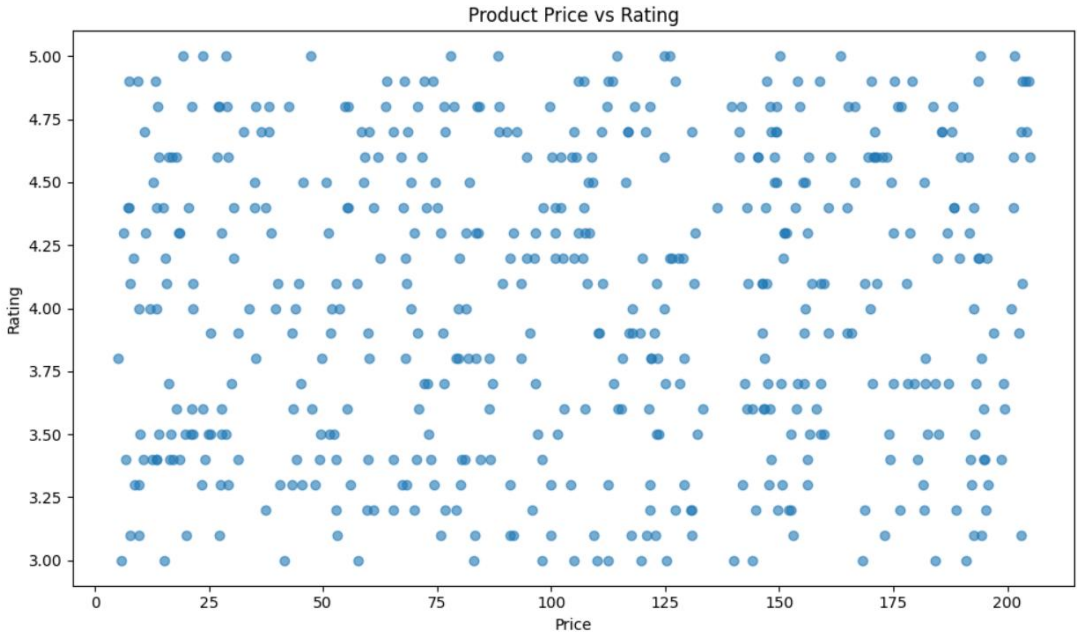


Figure 8. Product Price vs Rating

The x-axis shows Price and the y-axis shows Rating. Every point represents one product. The plot is used to inspect visually whether price and rating appear to be related. In this chart the points are spread over the entire price range for every rating value between 3.0 and 5.0, and no clear upward or downward pattern is visible. This visual impression agrees with the small correlation coefficient discussed in Section 7.6.

6.6 Vendor-wise Average Price Bar Chart

```
plt.figure(figsize=(12, 6))

vendor_price.plot(kind="bar")

plt.title("Average Product Price by Vendor")
plt.xlabel("Vendor")
plt.ylabel("Average Price")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

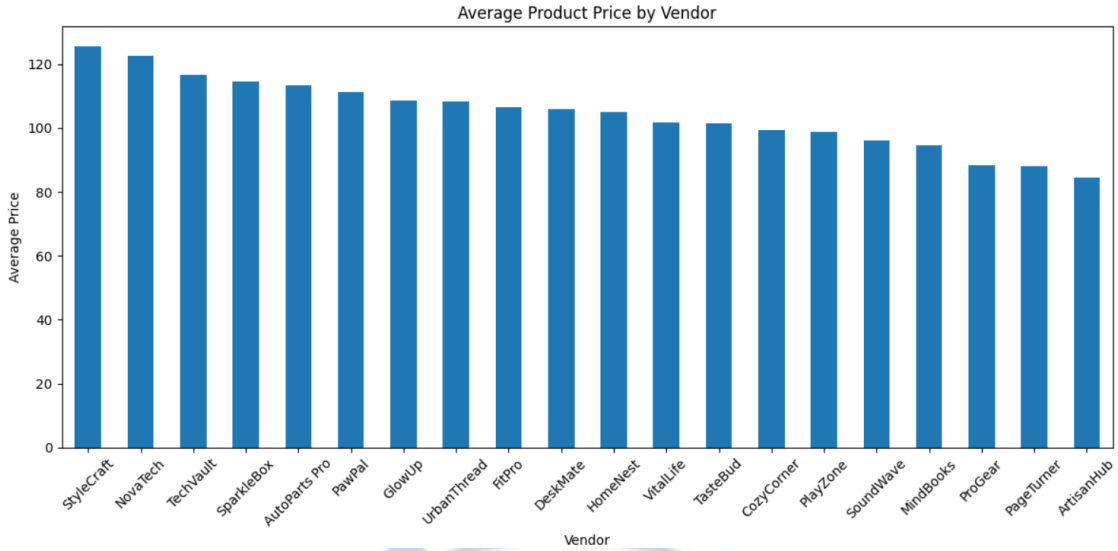


Figure 9. Average Product Price by Vendor

The chart compares the average product price of the vendors. As in the category chart, the vendors are sorted from the highest to the lowest average. The chart contains 20 vendors. The values behind the bars are tabulated in Section 7.7.

6.7 Interpretation of Visualisations

Taken together, the charts show that the prices in this dataset cover a wide range and are spread rather evenly instead of being concentrated around one value; that the average prices of categories and vendors differ, but only moderately; that the three price groups contain comparable numbers of products; and that price and rating do not show an obvious visual relationship. These statements describe the dataset that was scraped from the practice website; they should not be generalised to real marketplaces.

CHAPTER 7
ANALYSIS OF RESULTS AND DISCUSSION

7.1 Overview of Analysis

This chapter reports the numerical results of the analysis. All numbers in this chapter are the values printed by the project's code, as shown in the notebook output screenshots of Chapter 8. They describe the 500 records scraped from Scraping Sandbox. A few additional values, such as the range, are simple differences calculated from the printed values and are marked as derived.

7.2 Overall Price Statistics

Statistic	Value
Total products (count)	500
Mean (average) price	104.10 (104.100060)
Standard deviation	58.274374
Minimum price	5.12
25% quartile	55.305
Median price (50%)	105.23
75% quartile	152.625
Maximum price	204.86
Price range (maximum − minimum, derived)	199.74

Table 8. Overall price statistics (output of describe() and the summary table)

The average price (104.10) and the median price (105.23) are close to each other, which suggests that the prices are distributed fairly evenly around the centre and are not strongly pulled in one direction by a few extreme values. The standard deviation of about 58.27 is large compared with the average, and the quartiles (55.305 and 152.625) show that the middle half of the products lies across a wide band of prices. This agrees with the wide, spread-out histogram of Section 6.2.

7.3 Cheapest and Most Expensive Products

	Product_Name	Category	Vendor	Price	Rating
Cheapest	Organic Eye Cream	Beauty	GlowUp	5.12	3.8
Most expensive	Classic Joggers	Clothing	TechVault	204.86	4.6

Table 9. Cheapest and most expensive product

Index	Product_Name	Category	Vendor	Price	Rating
231	Classic Joggers	Clothing	TechVault	204.86	4.6
221	Organic Jump Starter	Automotive	CozyCorner	204.67	4.9
38	Compact Canvas	Art	NovaTech	204.29	4.7
378	Compact Trail Mix	Food & Beverage	AutoParts Pro	203.99	4.9
412	Smart Hot Sauce	Food & Beverage	NovaTech	203.21	4.1
122	Handcrafted Business Book	Books	StyleCraft	203.17	4.9
172	Portable MIDI Controller	Music	NovaTech	203.13	3.1
171	Vintage Pull-Up Bar	Sports	GlowUp	202.91	4.7
337	Modern Perfume	Beauty	GlowUp	202.59	3.9
161	Premium Omega-3 Capsules	Health	GlowUp	201.63	5.0

Table 10. Top 10 most expensive products (the first column is the DataFrame row index)

Index	Product_Name	Category	Vendor	Price	Rating
313	Organic Eye Cream	Beauty	GlowUp	5.12	3.8
214	Portable Hair Pin	Jewelry	PageTurner	5.66	3.0
76	Modern Coffee Beans	Food & Beverage	DeskMate	6.22	4.3
176	Premium Spice Set	Food & Beverage	GlowUp	6.68	3.4
178	Professional Sun Shade	Automotive	HomeNest	7.31	4.4
233	Wireless Coloring Book	Books	TechVault	7.36	4.4
102	Smart Storage Basket	Home & Garden	TasteBud	7.55	4.9
173	Advanced Eye Cream	Beauty	PageTurner	7.70	4.1
376	Heavy-Duty Microphone	Music	FitPro	7.77	3.1
315	Deluxe SSD Drive	Electronics	ArtisanHub	8.33	4.2

Table 11. Top 10 cheapest products (the first column is the DataFrame row index)

The cheapest product is Organic Eye Cream (Beauty, GlowUp) at 5.12 and the most expensive product is Classic Joggers (Clothing, TechVault) at 204.86. The product names are the generated names used on the practice website; they should not be read as real market prices. The vendor GlowUp appears in both top-10 lists, and every product in the two lists has a rating between 3.0 and 5.0.

7.4 Category-wise Price Comparison

Category	Minimum	Maximum	Mean
Home & Garden	7.55	195.03	126.70
Health	13.40	201.63	114.52
Beauty	5.12	202.59	110.31
Toys	12.00	175.94	107.96
Office Supplies	9.44	195.45	107.03

Category	Minimum	Maximum	Mean
Food & Beverage	6.22	203.99	106.31
Music	7.77	203.13	105.93
Art	13.45	204.29	104.20
Jewelry	5.66	201.23	104.13
Automotive	7.31	204.67	103.60
Clothing	11.02	204.86	101.18
Books	7.36	203.17	100.48
Pet Supplies	15.00	192.71	97.32
Sports	13.39	202.91	95.14
Electronics	8.33	194.35	86.04

Table 12. Category-wise price statistics, sorted by mean price (from the average-price and min/max/mean outputs)

The dataset contains 15 categories. Home & Garden has the highest average price (126.70) and Electronics has the lowest (86.04). Most other categories have averages between about 95 and 115, so the difference between most categories is moderate. Within each category, however, the minimum and maximum prices cover a very wide range (the minimums are between 5.12 and 15.00 and the maximums between 175.94 and 204.86). A category average therefore says little about the price of an individual product.

7.5 Price Range Distribution

The classification uses `pd.cut()` with three equal-width bins. Using the printed minimum (5.12) and maximum (204.86), the width of each interval is (204.86 − 5.12) ÷ 3 ≈ 66.58, so the approximate limits of the groups are as follows (derived values; pandas widens the lowest limit very slightly so that the minimum value is included).

Group	Approximate price interval
Low Price	5.12 to about 71.70
Medium Price	about 71.70 to about 138.28
High Price	about 138.28 to 204.86

Table 13. Approximate limits of the three price groups (derived from the minimum and maximum price)

The bar chart in Section 6.4 shows that the three groups contain similar numbers of products. The exact counts per group are printed by `value_counts()` and are to be taken from that notebook output (see Section 8.15).

7.6 Price-Rating Correlation

The correlation between Price and Rating printed by the code is **0.05** (also shown in the final summary table). A coefficient of this size is very close to zero. It indicates that there is

practically no linear relationship between price and rating in this dataset: higher-priced products are not noticeably better rated, and cheaper products are not noticeably worse rated. The sign is slightly positive, but the magnitude is too small to suggest a meaningful trend. The scatter plot supports this reading. As always, correlation does not prove causation, and a coefficient near zero does not exclude non-linear relationships.

7.7 Vendor-wise Price Comparison

Rank	Vendor	Average price	Rank	Vendor	Average price
1	StyleCraft	125.47	11	HomeNest	105.01
2	NovaTech	122.57	12	VitalLife	101.70
3	TechVault	116.70	13	TasteBud	101.46
4	SparkleBox	114.63	14	CozyCorner	99.29
5	AutoParts Pro	113.48	15	PlayZone	98.78
6	PawPal	111.44	16	SoundWave	96.03
7	GlowUp	108.58	17	MindBooks	94.57
8	UrbanThread	108.25	18	ProGear	88.29
9	FitPro	106.60	19	PageTurner	88.18
10	DeskMate	106.06	20	ArtisanHub	84.58

Table 14. Vendor-wise average price (from the average-price-by-vendor output)

Twenty vendors appear in the dataset. StyleCraft has the highest average price (125.47) and ArtisanHub the lowest (84.58). As with categories, the vendor averages differ by tens of price units but remain within a moderate band around the overall average of 104.10.

7.8 Discussion of Findings

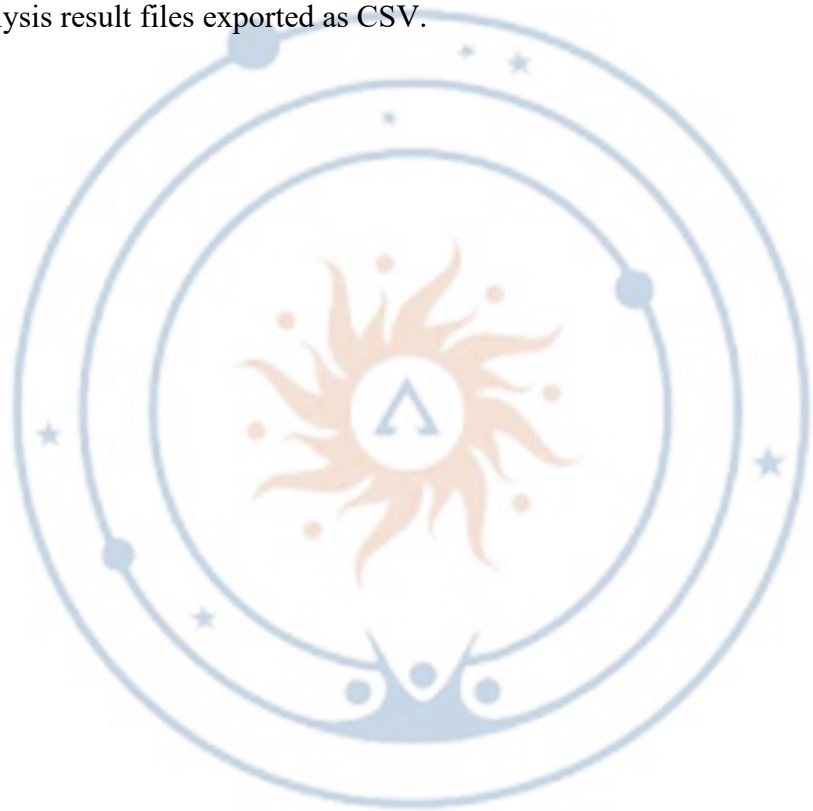
- The 500 prices range from 5.12 to 204.86, with a mean of 104.10 and a median of 105.23; the distribution is wide and fairly even.
- Category averages range from 86.04 (Electronics) to 126.70 (Home & Garden), and vendor averages from 84.58 (ArtisanHub) to 125.47 (StyleCraft).
- The three equal-width price groups contain comparable numbers of products.
- The price–rating correlation is 0.05, which shows no meaningful linear relationship.
- There are no missing values and no duplicate rows in the scraped dataset.

These findings describe a practice dataset whose values are generated for scraping practice. They show that the analysis pipeline works and produces meaningful descriptive summaries, but they should not be interpreted as statements about real product markets. The project does not use predictive modelling, and no future prices are estimated.

7.9 Project Outcomes

The outputs help a user to understand the product prices in the dataset quickly: the summary table gives the overall price level, the category and vendor tables show where averages are higher or lower, the top-10 tables show the extremes, the price-range chart shows how products are spread over price levels, and the correlation and scatter plot show whether rating is related to price. The implemented outcomes are:

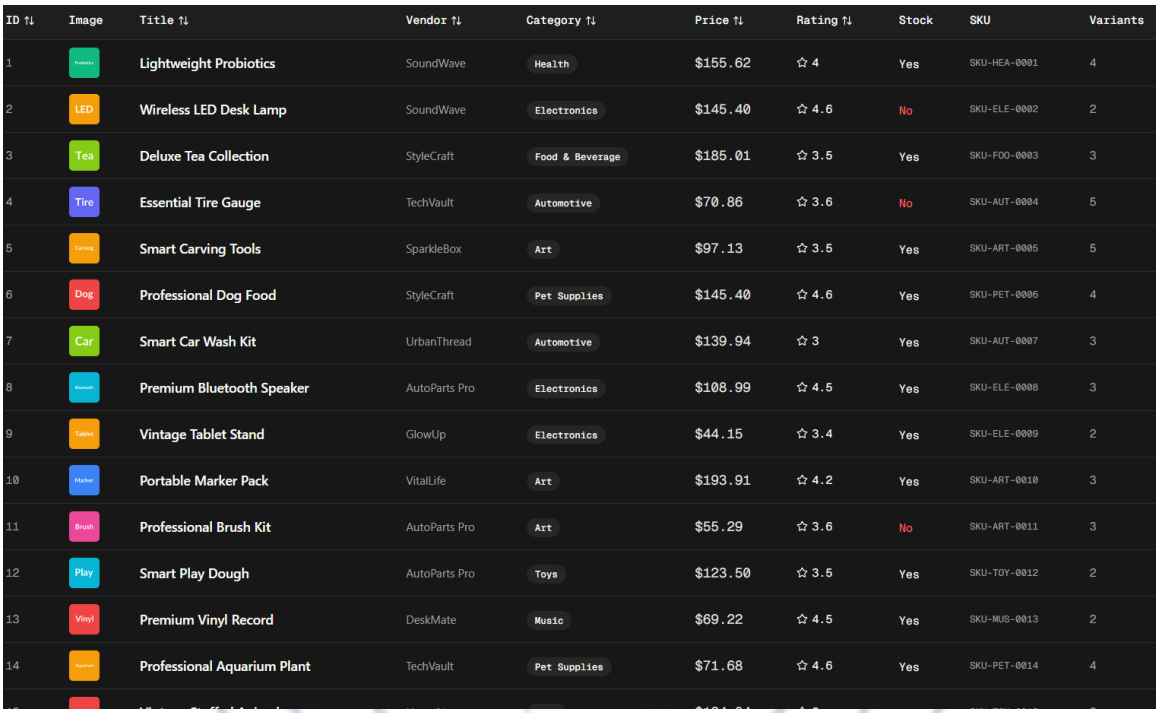
- 500 product records scraped from 20 pages and stored in a DataFrame.
- A validated and cleaned dataset saved as CSV and Excel files.
- Price statistics, category, vendor and price-range analyses and a price–rating correlation.
- Five charts and a final summary table.
- Four analysis result files exported as CSV.



CHAPTER 8
OUTPUT SCREENSHOTS

8.1 Scraping Sandbox Website Screenshot

This section and the following sections show the notebook outputs. Real screenshots from the project are used wherever they were available. Where a screenshot was not available, a clearly marked placeholder is provided, to be replaced in Microsoft Word with the actual screenshot.



ID ↑↓	Image	Title ↑↓	Vendor ↑↓	Category ↑↓	Price ↑↓	Rating ↑↓	Stock	SKU	Variants
1		Lightweight Probiotics	SoundWave	Health	$155.62	☆ 4	Yes	SKU-HEA-0001	4
2	LED	Wireless LED Desk Lamp	SoundWave	Electronics	$145.40	☆ 4.6	No	SKU-ELE-0002	2
3	Tea	Deluxe Tea Collection	StyleCraft	Food & Beverage	$185.01	☆ 3.5	Yes	SKU-FOO-0003	3
4	Tire	Essential Tire Gauge	TechVault	Automotive	$70.86	☆ 3.6	No	SKU-AUT-0004	5
5		Smart Carving Tools	SparkleBox	Art	$97.13	☆ 3.5	Yes	SKU-ART-0005	5
6	Dog	Professional Dog Food	StyleCraft	Pet Supplies	$145.40	☆ 4.6	Yes	SKU-PET-0006	4
7	Car	Smart Car Wash Kit	UrbanThread	Automotive	$139.94	☆ 3	Yes	SKU-AUT-0007	3
8		Premium Bluetooth Speaker	AutoParts Pro	Electronics	$108.99	☆ 4.5	Yes	SKU-ELE-0008	3
9		Vintage Tablet Stand	GlowUp	Electronics	$44.15	☆ 3.4	Yes	SKU-ELE-0009	2
10		Portable Marker Pack	VitalLife	Art	$193.91	☆ 4.2	Yes	SKU-ART-0010	3
11		Professional Brush Kit	AutoParts Pro	Art	$55.29	☆ 3.6	No	SKU-ART-0011	3
12	Play	Smart Play Dough	AutoParts Pro	Toys	$123.50	☆ 3.5	Yes	SKU-TOY-0012	2
13		Premium Vinyl Record	DeskMate	Music	$69.22	☆ 4.5	Yes	SKU-MUS-0013	2
14		Professional Aquarium Plant	TechVault	Pet Supplies	$71.68	☆ 4.6	Yes	SKU-PET-0014	4

Figure 10. The Data Table page of Scraping Sandbox (scrapingsandbox.com/data-table)

8.2 Browser Test Output

Cell 3 starts and closes headless Chromium and prints a confirmation message.

```python
from playwright.async_api import async_playwright

async def test_browser():

    async with async_playwright() as p:

        browser = await p.chromium.launch(
            headless=True
        )

        print("Browser started successfully!")

        await browser.close()

await test_browser()
```

```
Browser started successfully!
```

Figure 11. Cell 3: browser test code and output "Browser started successfully!"

8.3 Total Products Scraped Output

The last part of the scraping log shows "Rows found: 25" for the final pages and the total of 500 products.

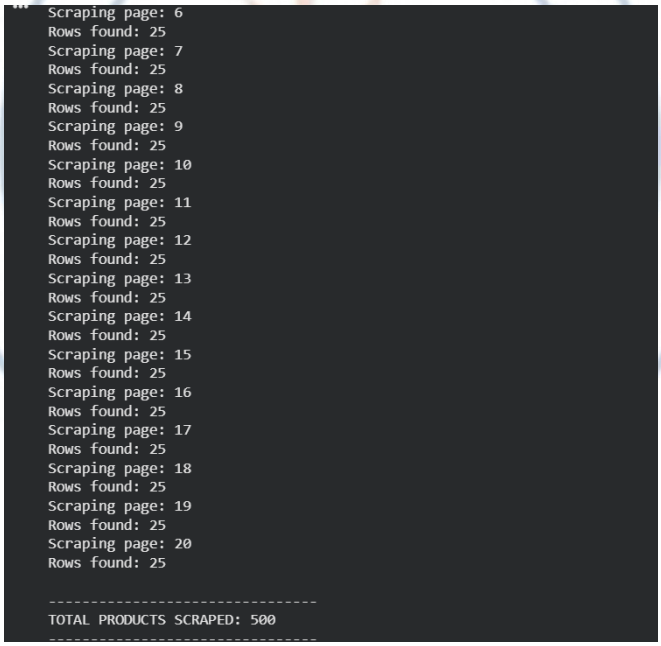


Figure 12. Cell 4 output: 25 rows found on each page and TOTAL PRODUCTS SCRAPED: 500

8.4 DataFrame Preview

The preview confirms the ten columns and shows that Price still contains the dollar symbol before cleaning.

	ID	Image	Product_Name	Vendor	Category	Price	Rating	Stock	SKU	Variants
0	1		Lightweight Probiotics	SoundWave	Health	$155.62	4	Yes	SKU-HEA-0001	4
1	2		Wireless LED Desk Lamp	SoundWave	Electronics	$145.40	4.6	No	SKU-ELE-0002	2
2	3		Deluxe Tea Collection	StyleCraft	Food & Beverage	$185.01	3.5	Yes	SKU-FOO-0003	3
3	4		Essential Tire Gauge	TechVault	Automotive	$70.86	3.6	No	SKU-AUT-0004	5
4	5		Smart Carving Tools	SparkleBox	Art	$97.13	3.5	Yes	SKU-ART-0005	5

Figure 13. Cell 5 output: the first five rows of the DataFrame

8.5 Dataset Validation Output

The output reports 500 rows, 10 columns, zero missing values in every column and zero duplicate rows.

```
Number of rows: 500
Number of columns: 10

Column names:
['ID', 'Image', 'Product_Name', 'Vendor', 'Category', 'Price', 'Rating', 'Stock', 'SKU', 'Variants']

Missing values:
ID              0
Image           0
Product_Name    0
Vendor          0
Category        0
Price           0
Rating          0
Stock           0
SKU             0
Variants        0
dtype: int64

Duplicate rows:
0
```

Figure 14. Cell 6 output: rows, columns, column names, missing values and duplicate rows

8.6 Data Cleaning Output

Cell 7 removes "$" and "," from Price and converts Price, Rating and Variants to numbers.

	ID	Image	Product_Name	Vendor	Category	Price	Rating	Stock	SKU	Variants
0	1		Lightweight Probiotics	SoundWave	Health	155.62	4.0	Yes	SKU-HEA-0001	4
1	2		Wireless LED Desk Lamp	SoundWave	Electronics	145.40	4.6	No	SKU-ELE-0002	2
2	3		Deluxe Tea Collection	StyleCraft	Food & Beverage	185.01	3.5	Yes	SKU-FOO-0003	3
3	4		Essential Tire Gauge	TechVault	Automotive	70.86	3.6	No	SKU-AUT-0004	5
4	5		Smart Carving Tools	SparkleBox	Art	97.13	3.5	Yes	SKU-ART-0005	5

Figure 15. Cell 7 output: the DataFrame after cleaning Price, Rating and Variants

8.7 Data Types Output

Cell 8 prints df.dtypes to confirm the converted data types.

```
ID              object
Image           object
Product_Name    object
Vendor          object
Category        object
Price          float64
Rating         float64
Stock           object
SKU             object
Variants         int64
dtype: object
```

Figure 16. Cell 8 output: data types of all columns after cleaning

8.8 Dataset Saving Output

The CSV and Excel files are written and the confirmation message "Dataset saved successfully!" is shown.

```
df.to_csv("scraped_product_data.csv", index=False)

df.to_excel("scraped_product_data.xlsx", index=False)

print("Dataset saved successfully!")
```

```
Dataset saved successfully!
```

Figure 17. Cell 9 output: dataset saved to CSV and Excel files

8.9 Price Statistics Output

The output gives an average price of 104.1, a minimum of 5.12, a maximum of 204.86 and a median of 105.23.

```
========== PRODUCT PRICE ANALYSIS ==========
Average Price   : 104.1
Minimum Price   : 5.12
Maximum Price   : 204.86
Median Price    : 105.23
```

Figure 18. Part B, step 3: average, minimum, maximum and median price

```
========== PRICE STATISTICS ==========
count    500.000000
mean     104.100060
std       58.274374
min        5.120000
25%       55.305000
50%      105.230000
75%      152.625000
max      204.860000
Name: Price, dtype: float64
```

Figure 19. Part B, step 4: output of df["Price"].describe()

8.10 Cheapest Product Output

The cheapest product is Organic Eye Cream (Beauty, GlowUp) with a price of 5.12 and a rating of 3.8.

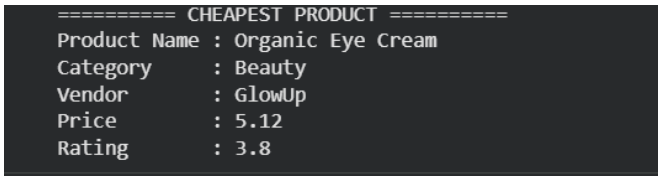


Figure 20. Part B, step 5: cheapest product

8.11 Most Expensive Product Output

The most expensive product is Classic Joggers (Clothing, TechVault) with a price of 204.86 and a rating of 4.6.

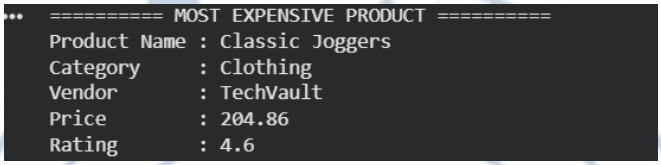


Figure 21. Part B, step 6: most expensive product

8.12 Category Price Analysis Output

The categories are sorted from the highest average price (Home & Garden) to the lowest (Electronics).

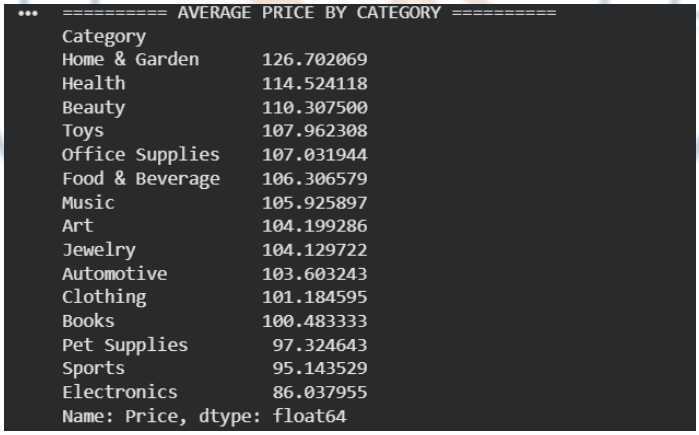


Figure 22. Part B, step 8: average price by category

Category	min	max	mean
Art	13.45	204.29	104.20
Automotive	7.31	204.67	103.60
Beauty	5.12	202.59	110.31
Books	7.36	203.17	100.48
Clothing	11.02	204.86	101.18
Electronics	8.33	194.35	86.04
Food & Beverage	6.22	203.99	106.31
Health	13.40	201.63	114.52
Home & Garden	7.55	195.03	126.70
Jewelry	5.66	201.23	104.13
Music	7.77	203.13	105.93
Office Supplies	9.44	195.45	107.03
Pet Supplies	15.00	192.71	97.32
Sports	13.39	202.91	95.14
Toys	12.00	175.94	107.96

Figure 23. Part B, step 10: minimum, maximum and mean price by category

8.13 Top 10 Expensive Products Output

The table lists the ten highest-priced products with their category, vendor, price and rating.

========== TOP 10 MOST EXPENSIVE PRODUCTS ==========

	Product_Name	Category	Vendor	Price	Rating
231	Classic Joggers	Clothing	TechVault	204.86	4.6
221	Organic Jump Starter	Automotive	CozyCorner	204.67	4.9
38	Compact Canvas	Art	NovaTech	204.29	4.7
378	Compact Trail Mix	Food & Beverage	AutoParts Pro	203.99	4.9
412	Smart Hot Sauce	Food & Beverage	NovaTech	203.21	4.1
122	Handcrafted Business Book	Books	StyleCraft	203.17	4.9
172	Portable MIDI Controller	Music	NovaTech	203.13	3.1
171	Vintage Pull-Up Bar	Sports	GlowUp	202.91	4.7
337	Modern Perfume	Beauty	GlowUp	202.59	3.9
161	Premium Omega-3 Capsules	Health	GlowUp	201.63	5.0

Figure 24. Part B, step 11: top 10 most expensive products

8.14 Top 10 Cheapest Products Output

The table lists the ten lowest-priced products with their category, vendor, price and rating.

========== TOP 10 CHEAPEST PRODUCTS ==========

	Product_Name	Category	Vendor	Price	Rating
313	Organic Eye Cream	Beauty	GlowUp	5.12	3.8
214	Portable Hair Pin	Jewelry	PageTurner	5.66	3.0
76	Modern Coffee Beans	Food & Beverage	DeskMate	6.22	4.3
176	Premium Spice Set	Food & Beverage	GlowUp	6.68	3.4
178	Professional Sun Shade	Automotive	HomeNest	7.31	4.4
233	Wireless Coloring Book	Books	TechVault	7.36	4.4
102	Smart Storage Basket	Home & Garden	TasteBud	7.55	4.9
173	Advanced Eye Cream	Beauty	PageTurner	7.70	4.1
376	Heavy-Duty Microphone	Music	FitPro	7.77	3.1
315	Deluxe SSD Drive	Electronics	ArtisanHub	8.33	4.2

Figure 25. Part B, step 12: top 10 cheapest products

8.15 Price Range Analysis Output

Step 13 prints the number of products in each of the three price groups created with pd.cut().

```
========== PRICE RANGE ANALYSIS ==========
Price_Range
High Price      172
Low Price       166
Medium Price    162
Name: count, dtype: int64
```

Figure 26. Part B, step 13 output: number of products in each price range

8.16 Correlation Output

The printed correlation between Price and Rating is 0.05.

```
Correlation between Price and Rating: 0.05
```

Figure 27. Part B, step 16: correlation between price and rating

8.17 Vendor Price Analysis Output

The vendors are sorted from the highest average price (StyleCraft) to the lowest (ArtisanHub).

```
Vendor
StyleCraft      125.472857
NovaTech        122.567097
TechVault       116.704000
SparkleBox      114.627059
AutoParts Pro   113.475517
PawPal          111.439444
GlowUp          108.580769
UrbanThread     108.254545
FitPro          106.599474
DeskMate        106.058148
HomeNest        105.006667
VitalLife       101.699615
TasteBud        101.462812
CozyCorner       99.287391
PlayZone         98.781364
SoundWave        96.026364
MindBooks        94.567391
ProGear          88.293043
PageTurner       88.182571
ArtisanHub       84.578750
Name: Price, dtype: float64
```

Figure 28. Part B, step 17: average price by vendor

8.18 Final Summary Table

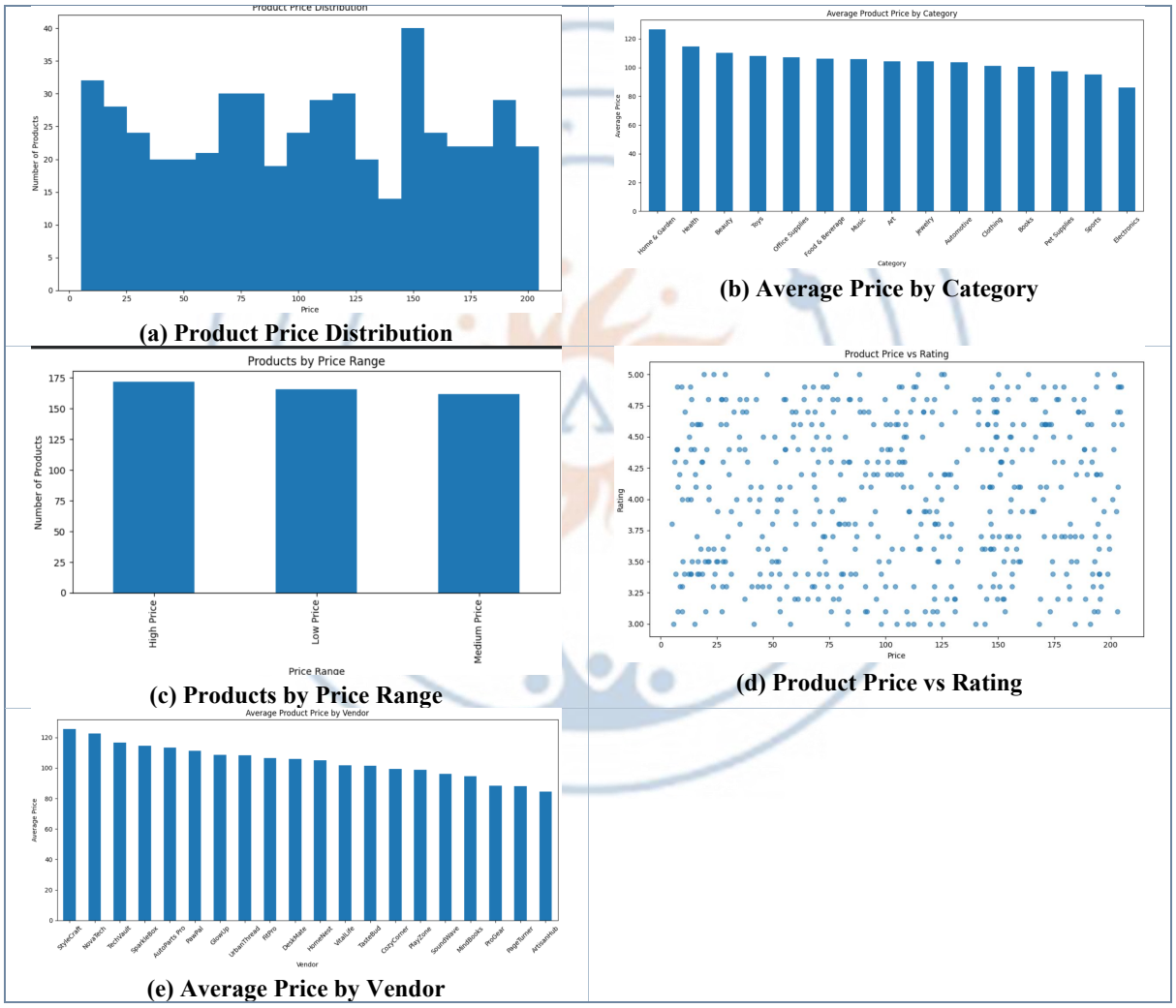
The table lists Total Products, Average Price, Minimum Price, Maximum Price, Median Price and Price-Rating Correlation.

	Metric	Value
0	Total Products	500.00
1	Average Price	104.10
2	Minimum Price	5.12
3	Maximum Price	204.86
4	Median Price	105.23
5	Price-Rating Correlation	0.05

Figure 29. Part B, step 19: final price analysis summary table

8.19 Visualisation Screenshots

The five charts generated by the analysis code are shown below in a gallery; each is also explained in Chapter 6.



(a) Product Price Distribution

(b) Average Price by Category

(c) Products by Price Range

(d) Product Price vs Rating

(e) Average Price by Vendor

Figure 30. Visualisation gallery of the five charts generated by the analysis code

CHAPTER 9
CONCLUSION, LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Web Scraping and Product Price Analysis System demonstrates an end-to-end workflow for collecting, cleaning, analysing, visualising and exporting product data. Using Playwright and a headless Chromium browser, the project scraped 500 product records from the 20 pages of the Scraping Sandbox Data Table, converted them into a Pandas DataFrame, validated them, cleaned the Price, Rating and Variants columns and saved the dataset. The analysis stage calculated price statistics, compared categories and vendors, classified products into three price groups, examined the relationship between price and rating, and exported the results. The project shows how the different parts of a data-analytics workflow depend on one another, and it produces results that are reproducible because the same code can be executed again.

9.2 Achievements

- Automated scraping of 500 records from 20 pages of a table whose pages are changed by buttons.
- A structured dataset with ten columns, validated for size, missing values and duplicates (500 rows, 10 columns, no missing values, no duplicate rows).
- Numeric conversion of Price, Rating and Variants.
- Price statistics, cheapest and most expensive product, category-wise and vendor-wise comparisons, top-10 lists, price-range classification and price–rating correlation.
- Five Matplotlib visualisations and a final summary table.
- CSV and Excel export of the cleaned dataset and CSV export of four analysis tables.

9.3 Limitations

- The implementation uses one practice website, Scraping Sandbox; it does not scrape any commercial e-commerce site.
- The data represents only the records available on that website at the time of scraping; the results describe this dataset and cannot be generalised to real marketplaces.
- The analysis focuses on price, rating, category and vendor; other attributes such as stock, SKU and variants are collected but not analysed.
- No predictive model is implemented, and no future prices are estimated.
- No live commercial product-comparison system is implemented.
- The pagination logic depends on the structure of the page (buttons named by page number, 20 pages, a wait of 500 ms), so a change in the page structure would require changes in the code.

- The code has no explicit error handling or logging.

9.4 Future Enhancements

The following are suggestions for future work. They are **not** implemented in the present project.

- Supporting additional data sources, where scraping is permitted.
- Scheduled, repeated data collection.
- Tracking price changes over time.
- Interactive dashboards for exploring the data.
- More advanced statistical analysis.
- Additional product-comparison features.
- Improved error handling and logging.

CHAPTER 10
REFERENCES

10.1 References

[1] Python Software Foundation, “Python 3 Documentation.” https://docs.python.org/3/

[2] Pandas Development Team, “pandas Documentation.” https://pandas.pydata.org/docs/

[3] Microsoft, “Playwright for Python – Documentation.” https://playwright.dev/python/

[4] Matplotlib Development Team, “Matplotlib Documentation.” https://matplotlib.org/stable/

[5] OpenPyXL, “openpyxl Documentation.” https://openpyxl.readthedocs.io/

[6] Google, “Google Colaboratory.” https://colab.research.google.com/

[7] Scraping Sandbox, “Data Table” (data source of this project). https://scrapingsandbox.com/data-table

CHAPTER 11
APPENDIX — SOURCE CODE

11.1 Part A: Web Scraping and Dataset Creation

The following listings are the complete, unmodified notebook cells of Part A.

Cell 1: Install

```
!pip install playwright openpyxl
!playwright install chromium
```

Cell 2: Install Chromium Dependencies

```
!python -m playwright install --with-deps chromium
```

Cell 3: Test The Browser

```
from playwright.async_api import async_playwright

async def test_browser():

    async with async_playwright() as p:

        browser = await p.chromium.launch(
            headless=True
        )

        print("Browser started successfully!")

        await browser.close()

await test_browser()
```

Cell 4: Scrape All 500 Products

```
import pandas as pd
from playwright.async_api import async_playwright

async def scrape_products():

    url = "https://scrapingsandbox.com/data-table"

    all_products = []

    async with async_playwright() as p:

        browser = await p.chromium.launch(headless=True)

        page = await browser.new_page()

        await page.goto(url, wait_until="domcontentloaded")

        await page.locator("table").wait_for()
```

```
        print("Website loaded successfully!")

        # There are 20 pages
        for page_number in range(1, 21):

            print("Scraping page:", page_number)

            # Click page number
            if page_number > 1:

                button = page.get_by_role(
                    "button",
                    name=str(page_number),
                    exact=True
                )

                await button.click()

                # Wait for table to update
                await page.wait_for_timeout(500)

            # Get table rows
            rows = page.locator("table tbody tr")

            row_count = await rows.count()

            print("Rows found:", row_count)

            # Extract every row
            for i in range(row_count):

                cells = rows.nth(i).locator("td")

                cell_count = await cells.count()

                values = []

                for j in range(cell_count):

                    value = await cells.nth(j).inner_text()

                    values.append(value.strip())

                if len(values) >= 10:

                    all_products.append(values)

        await browser.close()

    return all_products


all_products = await scrape_products()

print("\n--------------------------------")
print("TOTAL PRODUCTS SCRAPED:", len(all_products))
print("--------------------------------")
```

Cell 5: Convert Scraped Data Into A DataFrame

```
columns = [
    "ID",
    "Image",
    "Product_Name",
    "Vendor",
    "Category",
    "Price",
    "Rating",
    "Stock",
    "SKU",
    "Variants"
]

df = pd.DataFrame(all_products, columns=columns)

print("Dataset shape:", df.shape)

df.head()
```

Cell 6: Check The Scraped Dataset

```
print("Number of rows:", len(df))
print("Number of columns:", len(df.columns))

print("\nColumn names:")
print(df.columns.tolist())

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:")
print(df.duplicated().sum())
```

Cell 7: Clean Price And Rating

```
df["Price"] = (
    df["Price"]
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .astype(float)
)

df["Rating"] = pd.to_numeric(
    df["Rating"],
    errors="coerce"
)

df["Variants"] = pd.to_numeric(
    df["Variants"],
    errors="coerce"
)

df.head()
```

Cell 8: Verify The Cleaned Data

```
print(df.dtypes)
```

Cell 9: Save The Actual Scraped Dataset

```
df.to_csv("scraped_product_data.csv", index=False)

df.to_excel("scraped_product_data.xlsx", index=False)

print("Dataset saved successfully!")
```

Cell 10: Download The Dataset

```
from google.colab import files

files.download("scraped_product_data.csv")
```

11.2 Part B: Product Price Analysis

The following listings are the complete, unmodified analysis steps of Part B. Step numbers follow the numbering of the source notebook.

Step 1: Load The Dataset

```
import pandas as pd

df = pd.read_csv("scraped_product_data.csv")

print("Dataset loaded successfully!")
print("Number of products:", len(df))
print("Number of columns:", len(df.columns))

df.head()
```

Step 2: Clean The Price Column

```
df["Price"] = (
    df["Price"]
    .astype(str)
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
)

df["Price"] = pd.to_numeric(df["Price"], errors="coerce")

print(df["Price"].head())
```

Step 3: Basic Price Analysis

```
print("========== PRODUCT PRICE ANALYSIS ==========")

print("Average Price   :", round(df["Price"].mean(), 2))
print("Minimum Price   :", round(df["Price"].min(), 2))
print("Maximum Price   :", round(df["Price"].max(), 2))
print("Median Price    :", round(df["Price"].median(), 2))
```

Step 4: Complete Price Statistics

```
price_statistics = df["Price"].describe()

print("========== PRICE STATISTICS ==========")
print(price_statistics)
```

Step 5: Find The Cheapest Product

```
cheapest_product = df.loc[df["Price"].idxmin()]

print("========== CHEAPEST PRODUCT ==========")
print("Product Name :", cheapest_product["Product_Name"])
print("Category     :", cheapest_product["Category"])
print("Vendor       :", cheapest_product["Vendor"])
print("Price        :", cheapest_product["Price"])
print("Rating       :", cheapest_product["Rating"])
```

Step 6: Find The Most Expensive Product

```
expensive_product = df.loc[df["Price"].idxmax()]

print("========== MOST EXPENSIVE PRODUCT ==========")
print("Product Name :", expensive_product["Product_Name"])
print("Category     :", expensive_product["Category"])
print("Vendor       :", expensive_product["Vendor"])
print("Price        :", expensive_product["Price"])
print("Rating       :", expensive_product["Rating"])
```

Step 7: Price Distribution

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

plt.hist(df["Price"], bins=20)

plt.title("Product Price Distribution")
plt.xlabel("Price")
plt.ylabel("Number of Products")

plt.tight_layout()
plt.show()
```

Step 8: Average Price By Category

```
category_price = (
    df.groupby("Category")["Price"]
      .mean()
      .sort_values(ascending=False)
)

print("========== AVERAGE PRICE BY CATEGORY ==========")
print(category_price)
```

Step 9: Category Price Bar Chart

```
plt.figure(figsize=(12, 6))

category_price.plot(kind="bar")

plt.title("Average Product Price by Category")
plt.xlabel("Category")
plt.ylabel("Average Price")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Step 10: Minimum And Maximum Price By Category

```
category_price_range = df.groupby("Category")["Price"].agg(
    ["min", "max", "mean"]
)

category_price_range = category_price_range.round(2)

print("========== CATEGORY PRICE RANGE ==========")
print(category_price_range)
```

Step 11: Top 10 Most Expensive Products

```
top_expensive = (
    df.sort_values("Price", ascending=False)
      [["Product_Name", "Category", "Vendor", "Price", "Rating"]]
      .head(10)
)

print("========== TOP 10 MOST EXPENSIVE PRODUCTS ==========")

top_expensive
```

Step 12: Top 10 Cheapest Products

```
top_cheapest = (
    df.sort_values("Price", ascending=True)
      [["Product_Name", "Category", "Vendor", "Price", "Rating"]]
      .head(10)
)

print("========== TOP 10 CHEAPEST PRODUCTS ==========")

top_cheapest
```

Step 13: Price Range Classification

```
min_price = df["Price"].min()
max_price = df["Price"].max()

df["Price_Range"] = pd.cut(
    df["Price"],
    bins=3,
    labels=["Low Price", "Medium Price", "High Price"]
)

print("========== PRICE RANGE ANALYSIS ==========")

print(df["Price_Range"].value_counts())
```

Step 14: Price Range Chart

```
price_range_count = df["Price_Range"].value_counts()

plt.figure(figsize=(8, 5))

price_range_count.plot(kind="bar")

plt.title("Products by Price Range")
plt.xlabel("Price Range")
plt.ylabel("Number of Products")

plt.tight_layout()
plt.show()
```

Step 15: Price Vs Rating

```
plt.figure(figsize=(10, 6))

plt.scatter(
    df["Price"],
    df["Rating"],
    alpha=0.6
)

plt.title("Product Price vs Rating")
plt.xlabel("Price")
plt.ylabel("Rating")

plt.tight_layout()
plt.show()
```

Step 16: Price-Rating Correlation

```
price_rating_correlation = df["Price"].corr(df["Rating"])

print(
    "Correlation between Price and Rating:",
    round(price_rating_correlation, 3)
)
```

Step 17: Vendor-Wise Average Price

```
vendor_price = (
    df.groupby("Vendor")["Price"]
      .mean()
      .sort_values(ascending=False)
)

print("========== AVERAGE PRICE BY VENDOR ==========")
print(vendor_price)
```

Step 18: Vendor Price Chart

```
plt.figure(figsize=(12, 6))

vendor_price.plot(kind="bar")

plt.title("Average Product Price by Vendor")
plt.xlabel("Vendor")
plt.ylabel("Average Price")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()
```

Step 19: Create A Final Price Analysis Table

```
price_summary = pd.DataFrame({
    "Metric": [
        "Total Products",
        "Average Price",
        "Minimum Price",
        "Maximum Price",
        "Median Price",
        "Price-Rating Correlation"
    ],

    "Value": [
        len(df),
        round(df["Price"].mean(), 2),
        round(df["Price"].min(), 2),
        round(df["Price"].max(), 2),
        round(df["Price"].median(), 2),
        round(df["Price"].corr(df["Rating"]), 3)
    ]
})

price_summary
```

Step 20: Save The Price Analysis Results

```
price_summary.to_csv(
    "product_price_analysis_summary.csv",
    index=False
)

category_price_range.to_csv(
    "category_price_analysis.csv"
)

top_expensive.to_csv(
    "top_10_expensive_products.csv",
    index=False
)

top_cheapest.to_csv(
    "top_10_cheapest_products.csv",
    index=False
)

print("Price analysis files saved successfully!")
```